Mathematics

Arthur Li

May 7, 2026

Contents

1 Fundamentals of Higher Mathematics **1**
1.1 Single Variable Calculus . . . 1

2 Linear Algebra **4**
2.1 Structural Linear Algebra . . . 5
2.1.1 Vector Spaces and Linear Structure . . . 5
2.1.2 Linear Maps and Representations . . . 10

3 Time Series Analysis **14**
3.1 Time Series Analysis . . . 14
3.1.1 Stationary Time Series . . . 14
3.1.2 Univariate Time Series Models . . . 14

4 Mathematical and Statistical Foundation **17**
4.1 Classical Machine Learning . . . 18
4.1.1 Ensemble Methods . . . 18
4.2 Deep Learning . . . 19
4.2.1 Deep Feedforward Networks . . . 19
4.3 Regularisation for Deep Learning . . . 26
4.3.1 Optimisation for Deep Learning . . . 31
4.3.2 Convolutional Neural Networks . . . 40

1 Fundamentals of Higher Mathematics

Generally covers four areas: single-variable calculus, linear algebra, multivariable calculus, and differential equations. As this is fundamentals, the focus is generally on computation instead of proofs.

1.1 Single Variable Calculus

Calculus is the branch of mathematics that studies change and accumulation. It has two main branches:

i. Differential Calculus: Concerned with the rate at which things change (e.g. velocity).

ii. Integral Calculus: Concerned with accumulation (e.g. area under a curve).

Calculus underpins physics, engineering, economics, biology, and more.

Definition 1.1.1. The *derivative* is defined as:

$$\lim_{\Delta x \to 0} \frac{\Delta f}{\Delta x} = \lim_{\Delta x \to 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x} = f'(x_0)$$

Definition 1.1.2. A function $f(x)$ is continuous at x_0 when

$$\lim_{x \to x_0} f(x) = f(x_0)$$

Definition 1.1.3. The *right-hand* and *left-hand* limits are as follows:

i. *Right-Hand Limit*: $\lim_{x \to x_0^+} f(x)$ means $\lim_{x \to x_0} f(x)$ for $x > x_0$

ii. *Left-Hand Limit*: $\lim_{x \to x_0^-} f(x)$ means $\lim_{x \to x_0} f(x)$ for $x < x_0$

Definition 1.1.4. A *removable discontinuity* is where

$$\lim_{x \to x_0^+} f(x) = \lim_{x \to x_0^-} f(x)$$

but not at $f(x_0)$, or if $f(x_0)$ is undefined.

Definition 1.1.5. A *jump discontinuity* is where $\lim_{x \to x_0^+} f(x)$ and $\lim_{x \to x_0^-} f(x)$ exists, but not equal.

Definition 1.1.6. A *infinite discontinuity* is where $\lim_{x \to x_0^+} f(x) \neq \lim_{x \to x_0^-} f(x) \in \{+\infty, -\infty\}$

Theorem 1.1.7. *Differentiability Implies Continuity*
If f is differentiable at x_0, then f is continuous at x_0.

Proof.

$$\lim_{x \to x_0} (f(x) - f(x_0)) = \lim_{x \to x_0} \left[\frac{f(x) - f(x_0)}{x - x_0} \right] (x - x_0) = f'(x_0) \cdot 0 = 0$$

□

Definition 1.1.8. *Product Formula*
The derivative of product of two functions.

$$(u \cdot v)' = u' \cdot v + u \cdot v'$$

Definition 1.1.9. *Quotient Formula*
The derivate of ratio of two differentiable functions. Let $v(x) \neq 0$, then

$$\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2}$$

Definition 1.1.10. *Chain Rule*
The derivative of composition of two functions.

$$\frac{d}{dt} f(g(t)) = f'(g(t))g'(t)$$

May be used to differentiate implicitly defined functions.

Theorem 1.1.11. *Taylor's Theorem*
Let $k \geq 1$ be an integer and let function $f : \mathbb{R} \to \mathbb{R}$ be k-times differentiable at point $a \in \mathbb{R}$. Then there exists a function $h_k : \mathbb{R} \to \mathbb{R}$ such that

$$f(x) = \sum_{i=0}^{k} \frac{f^{(i)}(a)}{i!}(x-a)^i + h_k(x)(x-a)^k$$

where $\lim_{x \to a} h_k(x) = 0$ is the Peano form of the remainder.

Theorem 1.1.12. *Mean Value Theorem*
Let $f : [a, b] \to \mathbb{R}$ be a continuous function on closed interval $[a, b]$ and differentiable on open interval (a, b) where $a < b$. Then $\exists c \in (a, b)$ such that

$$f'(c) = \frac{f(b) - f(a)}{b - a}$$

Theorem 1.1.13. *First Fundamental Theorem of Calculus*
Let f be continuous real-valued function on closed interval $[a, b]$ Let F be function defined for all $x \in [a, b]$, by

$$F(x) = \int_a^x f(t)dt$$

Then F is uniformly continuous on $[a, b]$ and differentiable on open interval (a, b), and

$$F'(x) = f(x)$$

for all x in (a, b) so F is an antiderivative of f.

Remark 1.1.14. *Change of Variable*
If $f(x) = g(u(x))$, then $du = u'(x)dx$, hence

$$\int g(u)du = \int g(u(x))u'(x)dx = \int f(x)u'(x)dx$$

For definite integrals,

$$\int_{x_1}^{x_2} f(x)u'(x)dx = \int_{u_1}^{u_2} g(u)d(u)$$

where $u_1 = u(x_1), u_2 = u(x_2)$.

Theorem 1.1.15. *Second Fundamental Theorem of Calculus*
Let $f : \mathbb{R} \to \mathbb{R}$ on $[a, b]$ and F a continuous function on $[a, b]$ which is an antiderivative of f in (a, b),

$$F'(x) = f(x)$$

If f is Riemann integrable on $[a, b]$, then

$$\int_a^b f(x)dx = F(b) - F(a)$$

Remark 1.1.16. *Integration by Trigonometric Substitution*
Evaluate trigonometric integral, work backwards to original variable by using $\text{trig}(\text{trig}^{-1})$ with right-triangle.

i. $a^2 - x^2$ use $x = a \sin u$ and $dx = a \cos u \; du$
ii. $a^2 + x^2$ use $x = a \tan u$ and $dx = a \sec^2 u \; du$
iii. $x^2 - a^2$ use $x = a \sec u$ and $dx = a \sec u \tan u \; du$

Remark 1.1.17. *Systematic Integration of Rational Functions*
For a rational function P/Q (ratio of polynomials), by method of partial fractions, factor Q completely into linear and quadratic factors, and write P/Q as a sum of simpler terms.

i. If $Q(x) = (x-a)^m(x-b)^n$, then the expression is

$$\frac{A_1}{x-a} + \frac{A_2}{(x-a)^2} + \cdots + \frac{A_m}{(x-a)^m} + \frac{B_1}{x-b} + \frac{B_2}{(x-b)^2} + \cdots + \frac{B_n}{(x-b)^n}$$

ii. If there are quadratic factor such as $(Ax^2+Bx+C)^p$, then there are terms

$$\frac{a_1x+b_1}{Ax^2+Bx+C}+\frac{a_2x+b_2}{(Ax^2+Bx+C)^2}+\cdots+\frac{a_px+b_p}{(Ax^2+Bx+C)^p}$$

for each factor. To integrate, complete the square and make trigonometric substitution.

Remark 1.1.18. *Integration by Parts*

$$\int_b^a uv'\,dx = uv|_a^b - \int_a^b u'v\,dx$$

This is used when $u'v$ is simpler than uv' (often the case when u' is simpler than u).

Remark 1.1.19. *Arc-Length*
Given $ds=\sqrt{dx^2+dy^2}$, depending on whether to integrate with respect to x,t,y, this is

$$ds=\sqrt{1+\left(\frac{dy}{dx}\right)^2}\,dx;\quad ds=\sqrt{\left(\frac{dx}{dt}\right)^2+\left(\frac{dy}{dt}\right)^2}\,dt;\quad ds=\sqrt{\left(\frac{dx}{dy}\right)^2+1}\,dy$$

Remark 1.1.20. *Surface Area for Surface of Revolution*

i. Around x-axis: (requires formula for $y=y(x)$)

$$2\pi y\,ds = 2\pi y\sqrt{1+\left(\frac{dy}{dx}\right)^2}\,dx$$

ii. Around y-axis: (requires formula for $x=x(y)$)

$$2\pi x\,ds = 2\pi x\sqrt{\left(\frac{dx}{dy}\right)^2+1}\,dy$$

Remark 1.1.21. *Polar Coordinates*
Given $x=r\cos\theta$, $y=r\sin\theta$ (or more rarely $r=\sqrt{x^2+y^2}\theta$, $\theta=\tan^{-1}(y/x)$)

i. Find the polar equation for a curve from its equation in (x,y) variables by substitution.
ii. Sketch curves given in polar coordinates and find range of θ for integration.
iii. Integration area formula is then

$$\int_{\theta_1}^{\theta_2}\frac{1}{2}r^2\,d\theta$$

Theorem 1.1.22. *L'Hopital Rule*
For functions f and g defined on open interval I and differentiable on $I\backslash\{c\}$ for a accumulation point c of I, if

$$\lim_{x\to c}f(x)=\lim_{x\to c}g(x)=0\quad or\quad \pm\infty$$

and $g'(x)\neq 0$ for all $x\in I\backslash\{c\}$, and $\lim_{x\to c}\frac{f'(x)}{g'(x)}$ exists, then

$$\lim_{x\to c}\frac{f(x)}{g(x)}=\lim_{x\to c}\frac{f'(x)}{g'(x)}$$

2 Linear Algebra

Linear algebra is the branch of mathematics that studies vectors, matrices, and linear transformations. It provides the foundation for solving systems of equations and analysing multidimensional data. Its applications span across machine learning, computer graphics, physics, finance, and engineering.
References from **?** and **?**.

2.1 Structural Linear Algebra

This is the coordinate-free backbone. The purpose is to understand vector spaces and linear maps abstractly.

2.1.1 Vector Spaces and Linear Structure

The guiding idea is that a vector space is not a list of coordinates first. It is a space of objects that can be added and scaled, and coordinates only appear after a basis is chosen.

Definition 2.1.1. *Vector Space*
Let $\mathbb{F}$ be a field. A vector space over $\mathbb{F}$ is a set $\mathcal{V}$ together with operations of vector addition and scalar multiplication such that, for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathcal{V}$ and all $a, b \in \mathbb{F}$, the following axioms hold:

i. vector addition is commutative: $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$;
ii. vector addition is associative: $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$;
iii. there exists an additive identity $\mathbf{0} \in \mathcal{V}$ such that $\mathbf{0} + \mathbf{v} = \mathbf{v}$;
iv. for each $\mathbf{v} \in \mathcal{V}$ there exists an additive inverse $-\mathbf{v} \in \mathcal{V}$ such that $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$;
v. scalar multiplication has identity: $1\mathbf{v} = \mathbf{v}$;
vi. scalar multiplication is compatible with multiplication in $\mathbb{F}$: $(ab)\mathbf{v} = a(b\mathbf{v})$;
vii. scalar multiplication distributes over vector addition: $a(\mathbf{u} + \mathbf{v}) = a\mathbf{u} + a\mathbf{v}$;
viii. scalar multiplication distributes over addition in $\mathbb{F}$: $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$.

Definition 2.1.2. Let $\mathcal{V}$ be a vector space over $\mathbb{F}$. If $\mathcal{V} = \{\mathbf{0}\}$, then $\mathcal{V}$ is a *zero vector space*; if $\mathcal{V} \neq \{\mathbf{0}\}$, then $\mathcal{V}$ is a *nonzero vector space*.

Theorem 2.1.3. *Basic Consequences of Vector Space Axioms*
Let $\mathcal{V}$ be a vector space over $\mathbb{F}$. Then the additive identity and additive inverses are unique. Moreover, for all $a \in \mathbb{F}$ and $\mathbf{v} \in \mathcal{V}$, $0\mathbf{v} = \mathbf{0}$, $a\mathbf{0} = \mathbf{0}$, $(-1)\mathbf{v} = -\mathbf{v}$ and $a\mathbf{v} = \mathbf{0}$ if and only if $a = 0$ or $\mathbf{v} = \mathbf{0}$.

Proof. If $\mathbf{0}$ and $\mathbf{0}'$ are additive identities, then $\mathbf{0} = \mathbf{0} + \mathbf{0}' = \mathbf{0}'$, so the additive identity is unique.
If $\mathbf{w}$ and $\mathbf{w}'$ are additive inverses of $\mathbf{v}$, then $\mathbf{w} = \mathbf{w} + \mathbf{0} = \mathbf{w} + (\mathbf{v} + \mathbf{w}') = (\mathbf{w} + \mathbf{v}) + \mathbf{w}' = \mathbf{0} + \mathbf{w}' = \mathbf{w}'$, so additive inverses are unique. Since $0\mathbf{v} = (0 + 0)\mathbf{v} = 0\mathbf{v} + 0\mathbf{v}$, cancellation gives $0\mathbf{v} = \mathbf{0}$.
Similarly, $a\mathbf{0} = a(\mathbf{0} + \mathbf{0}) = a\mathbf{0} + a\mathbf{0}$, so $a\mathbf{0} = \mathbf{0}$. Also $\mathbf{v} + (-1)\mathbf{v} = 1\mathbf{v} + (-1)\mathbf{v} = (1 - 1)\mathbf{v} = 0\mathbf{v} = \mathbf{0}$, hence $(-1)\mathbf{v}$ is the additive inverse of $\mathbf{v}$, and therefore $(-1)\mathbf{v} = -\mathbf{v}$.
Finally, if $a = 0$ or $\mathbf{v} = \mathbf{0}$, then the preceding identities give $a\mathbf{v} = \mathbf{0}$. Conversely, suppose $a\mathbf{v} = \mathbf{0}$ and $a \neq 0$. Then a^{-1} exists in $\mathbb{F}$, and $\mathbf{v} = 1\mathbf{v} = (a^{-1}a)\mathbf{v} = a^{-1}(a\mathbf{v}) = a^{-1}\mathbf{0} = \mathbf{0}$.
Structural Commentary: These identities are the basic algebraic hygiene of vector spaces. They let later arguments use cancellation, zero vectors, and inverses without reopening the axioms each time. □

Key Insight: The vector space axioms force the zero and inverse behaviour to be rigid, not optional structure.

Corollary 2.1.4. *Let $\mathcal{V}$ be a vector space over $\mathbb{F}$. Then $(-1)\mathbf{v} = -\mathbf{v}$ for every $\mathbf{v} \in \mathcal{V}$.*

Proof. Let $\mathbf{v} \in \mathcal{V}$. We show that $(-1)\mathbf{v} + \mathbf{v} = \mathbf{0}$. By the vector space axioms and Theorem 2.1.3,

$$(-1)\mathbf{v} + \mathbf{v} = (-1)\mathbf{v} + 1\mathbf{v} = (-1 + 1)\mathbf{v} = 0\mathbf{v} = \mathbf{0}$$

Thus $(-1)\mathbf{v}$ is the additive inverse of $\mathbf{v}$, so $(-1)\mathbf{v} = -\mathbf{v}$.
Structural Commentary: Scalar multiplication by -1 is exactly the same operation as taking the additive inverse. □

Key Insight: Negation is not an extra operation; it is already encoded by the field action.

Definition 2.1.5. A *subspace* of a vector space $\mathcal{V}$ over $\mathbb{F}$ is a subset $\mathcal{U} \subseteq \mathcal{V}$ that is itself a vector space over $\mathbb{F}$ with the same vector addition and scalar multiplication as in $\mathcal{V}$.

Theorem 2.1.6. *Subspace Test*
Let $\mathcal{V}$ be a vector space over $\mathbb{F}$ and let $\mathcal{U}$ be a nonempty subset of $\mathcal{V}$. Then $\mathcal{U}$ is a subspace of $\mathcal{V}$ if and only if $c\mathbf{u} + \mathbf{v} \in \mathcal{U}$ for all $\mathbf{u}, \mathbf{v} \in \mathcal{U}$ and all $c \in \mathbb{F}$.

Proof.

($\Rightarrow$) If $\mathcal{U}$ is a subspace, then $c\mathbf{u} + \mathbf{v} \in \mathcal{U}$ for all $\mathbf{u}, \mathbf{v} \in \mathcal{U}$ and all $c \in \mathbb{F}$ because $\mathcal{U}$ is closed under scalar multiplication and vector addition.

($\Leftarrow$) Suppose that $c\mathbf{u}+\mathbf{v} \in \mathcal{U}$ whenever $\mathbf{u}, \mathbf{v} \in \mathcal{U}$ and $c \in \mathbb{F}$. We verify that $\mathcal{U}$ is a vector space with the operations inherited from $\mathcal{V}$. Since $\mathcal{U}$ is nonempty, let $\mathbf{u} \in \mathcal{U}$. By Corollary 2.1.4, $\mathbf{0} = (-1)\mathbf{u}+\mathbf{u} \in \mathcal{U}$. If $\mathbf{u} \in \mathcal{U}$, then $-\mathbf{u} = (-1)\mathbf{u}+\mathbf{0} \in \mathcal{U}$. If $\mathbf{u}, \mathbf{v} \in \mathcal{U}$ and $c \in \mathbb{F}$, then $c\mathbf{u} = c\mathbf{u}+\mathbf{0} \in \mathcal{U}$ and $\mathbf{u}+\mathbf{v} = 1\mathbf{u}+\mathbf{v} \in \mathcal{U}$. Thus $\mathcal{U}$ contains $\mathbf{0}$ and is closed under additive inverses, scalar multiplication, and vector addition; the remaining vector space axioms are inherited from $\mathcal{V}$. Hence $\mathcal{U}$ is a subspace.

Structural Commentary: The theorem compresses the subspace axioms into one closure condition. This is the practical test used whenever a subset is proposed as a linear space. □

Key Insight: To prove a subset is a subspace, prove it survives one affine-looking linear combination.

Definition 2.1.7. *Lists, Linear Combinations, and Span*
A *list* of vectors in $\mathcal{V}$ is a nonempty finite ordered sequence $\beta = (\mathbf{v}_1, \ldots, \mathbf{v}_m)$ in $\mathcal{V}$, denoted by a Greek letter.
If $\mathbf{v}_1, \ldots, \mathbf{v}_m \in \mathcal{V}$ and $a_1, \ldots, a_m \in \mathbb{F}$, then $a_1\mathbf{v}_1 + \cdots + a_m\mathbf{v}_m$ is a *linear combination* of $\mathbf{v}_1, \ldots, \mathbf{v}_m$.
The *span* of a nonempty subset $E \subseteq \mathcal{V}$ is $\operatorname{span}(E) := \{a_1\mathbf{v}_1 + \cdots + a_m\mathbf{v}_m : m \in \mathbb{N},\ a_j \in \mathbb{F},\ \mathbf{v}_j \in E\}$.
By convention, $\operatorname{span}(\emptyset) = \{\mathbf{0}\}$. If $\beta = (\mathbf{v}_1, \ldots, \mathbf{v}_m)$ is a list, then $\operatorname{span}(\beta) = \operatorname{span}(\mathbf{v}_1, \ldots, \mathbf{v}_m) := \operatorname{span}(\{\mathbf{v}_1, \ldots, \mathbf{v}_m\})$.

Theorem 2.1.8. *Basic Properties of Span*
Let $E \subseteq \mathcal{V}$. Then:

(*i*) $\operatorname{span}(E)$ *is a subspace of* $\mathcal{V}$.

(*ii*) $E \subseteq \operatorname{span}(E)$.

(*iii*) $E = \operatorname{span}(E)$ *if and only if* E *is a subspace of* $\mathcal{V}$.

(*iv*) $\operatorname{span}(\operatorname{span}(E)) = \operatorname{span}(E)$.

Consequently, $\operatorname{span}(E)$ *is the smallest subspace of* $\mathcal{V}$ *containing* E.

Proof.

(*i*) If $E = \emptyset$, then $\operatorname{span}(E) = \{\mathbf{0}\}$, so $\operatorname{span}(E)$ is a subspace. Suppose $E \neq \emptyset$. If $\mathbf{x}, \mathbf{y} \in \operatorname{span}(E)$ and $c \in \mathbb{F}$, then $\mathbf{x}$ and $\mathbf{y}$ are finite linear combinations of vectors in E, hence $c\mathbf{x}+\mathbf{y}$ is again a finite linear combination of vectors in E. Thus $c\mathbf{x}+\mathbf{y} \in \operatorname{span}(E)$, and Theorem 2.1.6 shows that $\operatorname{span}(E)$ is a subspace.

(*ii*) Each $\mathbf{v} \in E$ satisfies $\mathbf{v} = 1\mathbf{v} \in \operatorname{span}(E)$, so $E \subseteq \operatorname{span}(E)$.

(*iii*) If $E = \operatorname{span}(E)$, then (*i*) shows that E is a subspace. Conversely, if E is a subspace, then E is closed under finite linear combinations, so $\operatorname{span}(E) \subseteq E$; together with (*ii*) this gives $E = \operatorname{span}(E)$.

(*iv*) By (*i*), $\operatorname{span}(E)$ is a subspace. Applying (*iii*) to the subspace $\operatorname{span}(E)$ gives $\operatorname{span}(\operatorname{span}(E)) = \operatorname{span}(E)$.

If $\mathcal{U} \subseteq \mathcal{V}$ containing E, closure under vector addition and scalar multiplication forces every finite linear combination of vectors in E to lie in $\mathcal{U}$. Hence $\operatorname{span}(E) \subseteq \mathcal{U}$, so $\operatorname{span}(E)$ is the smallest subspace containing E.
Structural Commentary: Taking span is a closure operation. It contains the original set, produces a subspace, and becomes stable after one application. □

Key Insight: Span is the smallest linear environment forced by a set of vectors.

Proposition 2.1.9. *Monotonicity of Span*
Let $\mathcal{U}$ and $\mathcal{W}$ be subsets of $\mathcal{V}$. If $\mathcal{U} \subseteq \mathcal{W}$, then $\operatorname{span}(\mathcal{U}) \subseteq \operatorname{span}(\mathcal{W})$.

Proof. If $\mathcal{U} = \emptyset$, then $\operatorname{span}(\mathcal{U}) = \{\mathbf{0}\} \subseteq \operatorname{span}(\mathcal{W})$.
Suppose $\mathcal{U} \neq \emptyset$. Every vector in $\operatorname{span}(\mathcal{U})$ is a finite linear combination of vectors in $\mathcal{U}$; since $\mathcal{U} \subseteq \mathcal{W}$, the same expression is a finite linear combination of vectors in $\mathcal{W}$. Hence $\operatorname{span}(\mathcal{U}) \subseteq \operatorname{span}(\mathcal{W})$.
Structural Commentary: Enlarging the generating set can only enlarge the collection of linear combinations. □

Key Insight: Span is monotone with respect to inclusion of generating sets.

Definition 2.1.10. *Linear Independence*
A list $(\mathbf{v}_1, \ldots, \mathbf{v}_m)$ in $\mathcal{V}$ is linearly independent if

$$a_1\mathbf{v}_1 + \cdots + a_m\mathbf{v}_m = \mathbf{0}$$

implies $a_1 = \cdots = a_m = 0$. Otherwise the list is linearly dependent.

Lemma 2.1.11. *Linear Dependence Lemma*
If $(\mathbf{v}_1, \dots, \mathbf{v}_m)$ is linearly dependent and $\mathbf{v}_1 \neq \mathbf{0}$, then there exists $j \in \{2, \dots, m\}$ such that

$$\mathbf{v}_j \in \operatorname{span}(\mathbf{v}_1, \dots, \mathbf{v}_{j-1}),$$

and removing $\mathbf{v}_j$ does not change the span of the list.

Proof. Because the list is dependent, there exist scalars $a_1, \dots, a_m$, not all zero, such that

$$a_1\mathbf{v}_1 + \cdots + a_m\mathbf{v}_m = \mathbf{0}.$$

Choose j to be the largest index such that $a_j \neq 0$. Then

$$\mathbf{v}_j = -\frac{a_1}{a_j}\mathbf{v}_1 - \cdots - \frac{a_{j-1}}{a_j}\mathbf{v}_{j-1},$$

so $\mathbf{v}_j$ lies in the span of the preceding vectors. Any linear combination involving $\mathbf{v}_j$ can therefore be rewritten using only the remaining vectors, so removing $\mathbf{v}_j$ does not alter the span. *Structural Commentary*: A dependence relation identifies a redundant vector. Choosing the largest nonzero coefficient makes the redundancy compatible with the order of the list. □

Key Insight: Linear dependence means at least one vector can be discarded without losing span.

Theorem 2.1.12. *In a finite-dimensional vector space, the length of every linearly independent list is at most the length of every spanning list.*

Proof. Let $(\mathbf{u}_1, \dots, \mathbf{u}_m)$ be linearly independent and let $(\mathbf{w}_1, \dots, \mathbf{w}_n)$ span $\mathcal{V}$. Starting from the spanning list $(\mathbf{w}_1, \dots, \mathbf{w}_n)$, insert $\mathbf{u}_1, \dots, \mathbf{u}_m$ one at a time and, whenever dependence appears, remove a vector from the old spanning list using Lemma 2.1.11. After m steps we still have a spanning list of length n containing $\mathbf{u}_1, \dots, \mathbf{u}_m$, hence $m \leq n$. *Structural Commentary*: This comparison theorem is the counting engine behind dimension. It says independence and spanning place opposite pressure on list length. □

Key Insight: Independent lists cannot be longer than spanning lists in finite dimensions.

Definition 2.1.13. *Basis*
A basis of $\mathcal{V}$ is a list of vectors that is linearly independent and spans $\mathcal{V}$.

Proposition 2.1.14. *A list $(\mathbf{v}_1, \dots, \mathbf{v}_n)$ is a basis of $\mathcal{V}$ if and only if every $\mathbf{v} \in \mathcal{V}$ can be written uniquely as*

$$\mathbf{v} = a_1\mathbf{v}_1 + \cdots + a_n\mathbf{v}_n.$$

Structural Commentary: A basis is exactly a coordinate system: spanning gives existence and independence gives uniqueness.

Key Insight: Basis vectors are useful because they make every vector have one and only one coordinate description.

Theorem 2.1.15. *Reducing Spanning Lists*
Every spanning list in a finite-dimensional vector space can be reduced to a basis.

Proof. Repeatedly remove any vector that lies in the span of the preceding vectors. Lemma 2.1.11 shows that the span does not change, and when no such vector remains the list is linearly independent. The resulting list is therefore a basis. *Structural Commentary*: A spanning list may contain redundancy, but finite-dimensionality lets us remove it until only essential generators remain. □

Key Insight: Every finite spanning list hides a basis inside it.

Theorem 2.1.16. *Extending Independent Lists*
Every linearly independent list in a finite-dimensional vector space can be extended to a basis.

Proof. Take any finite spanning list and append its vectors one at a time to the independent list, keeping only those that are not already in the span. This preserves linear independence and eventually yields a spanning list, hence a basis. *Structural Commentary*: Independent lists are partial coordinate systems. Extension fills in the missing directions without destroying independence. □

Key Insight: Every finite-dimensional independent list can be completed to a basis.

Corollary 2.1.17. *Every finite-dimensional vector space has a basis, and every subspace of a finite-dimensional vector space is finite-dimensional. Structural Commentary: Finite-dimensional spaces and their subspaces admit finite coordinate descriptions.*

Key Insight: Finite-dimensionality passes downward to subspaces.

Theorem 2.1.18. *Basis Length Is Well-Defined*
Any two bases of a finite-dimensional vector space have the same length.

Proof. Each basis is linearly independent and spanning, so Theorem 2.1.12 applied in both directions gives equality of lengths. *Structural Commentary*: This result makes dimension well-defined. Without it, the length of a basis would depend on the chosen coordinates. □

Key Insight: Dimension is an intrinsic number, not an artifact of a particular basis.

Definition 2.1.19. *Dimension*
If $\mathcal{V}$ is finite-dimensional, its dimension is the common length of any basis of $\mathcal{V}$, denoted $\dim(\mathcal{V})$.

Proposition 2.1.20. *If $\mathcal{V}$ is finite-dimensional and $\mathcal{U} \subseteq \mathcal{V}$ is a subspace, then*

$$\dim(\mathcal{U}) \leq \dim(\mathcal{V}).$$

Moreover, a spanning list of length $\dim(\mathcal{V})$ is automatically a basis, and a linearly independent list of length $\dim(\mathcal{V})$ is automatically a basis. Structural Commentary: Dimension bounds the size of subspaces and identifies when a list is exactly the right size to be a basis.

Key Insight: In dimension n, either n independent vectors or n spanning vectors already give a basis.

Definition 2.1.21. *Sums of Subspaces*
If $\mathcal{U}_1, \ldots, \mathcal{U}_m$ are subspaces of $\mathcal{V}$, define

$$\mathcal{U}_1 + \cdots + \mathcal{U}_m := \{\mathbf{u}_1 + \cdots + \mathbf{u}_m : \mathbf{u}_j \in \mathcal{U}_j\}.$$

If every vector in this sum has a unique such decomposition, then the sum is direct and we write

$$\mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m.$$

Proposition 2.1.22. *Sum of Subspaces Is the Smallest Containing Subspace*
Suppose $\mathcal{U}_1, \ldots, \mathcal{U}_m$ are subspaces of $\mathcal{V}$. Then $\mathcal{U}_1 + \cdots + \mathcal{U}_m$ is the smallest subspace of $\mathcal{V}$ containing $\mathcal{U}_1, \ldots, \mathcal{U}_m$.

Proof. First, $\mathbf{0} \in \mathcal{U}_1 + \cdots + \mathcal{U}_m$. If $\mathbf{x} = \mathbf{u}_1 + \cdots + \mathbf{u}_m$ and $\mathbf{y} = \mathbf{v}_1 + \cdots + \mathbf{v}_m$ lie in $\mathcal{U}_1 + \cdots + \mathcal{U}_m$, then for any $c \in \mathbb{F}$,

$$c\mathbf{x} + \mathbf{y} = (c\mathbf{u}_1 + \mathbf{v}_1) + \cdots + (c\mathbf{u}_m + \mathbf{v}_m) \in \mathcal{U}_1 + \cdots + \mathcal{U}_m,$$

because each $\mathcal{U}_j$ is a subspace. By Theorem 2.1.6, $\mathcal{U}_1 + \cdots + \mathcal{U}_m$ is a subspace of $\mathcal{V}$.
Clearly each $\mathcal{U}_j$ is contained in $\mathcal{U}_1 + \cdots + \mathcal{U}_m$ by setting all other summands equal to $\mathbf{0}$. Conversely, if $\mathcal{W}$ is any subspace of $\mathcal{V}$ containing $\mathcal{U}_1, \ldots, \mathcal{U}_m$, then every sum $\mathbf{u}_1 + \cdots + \mathbf{u}_m$ with $\mathbf{u}_j \in \mathcal{U}_j$ lies in $\mathcal{W}$. Hence $\mathcal{U}_1 + \cdots + \mathcal{U}_m \subseteq \mathcal{W}$, so the sum is the smallest such subspace. *Structural Commentary*: Sums of subspaces behave like unions after enforcing linear closure. The smallest subspace containing several pieces is obtained by allowing finite sums of vectors from those pieces. □

Key Insight: The sum of subspaces is the linear closure of those subspaces taken together.

Proposition 2.1.23. *For subspaces $\mathcal{U}_1, \ldots, \mathcal{U}_m \subseteq \mathcal{V}$, the following are equivalent:*

i. $\mathcal{V} = \mathcal{U}_1 \oplus \cdots \oplus \mathcal{U}_m$;

ii. *every $\mathbf{v} \in \mathcal{V}$ has a unique representation $\mathbf{v} = \mathbf{u}_1 + \cdots + \mathbf{u}_m$ with $\mathbf{u}_j \in \mathcal{U}_j$;*

iii. *if $\mathbf{u}_1 + \cdots + \mathbf{u}_m = \mathbf{0}$ with $\mathbf{u}_j \in \mathcal{U}_j$, then $\mathbf{u}_1 = \cdots = \mathbf{u}_m = \mathbf{0}$.*

Structural Commentary: Direct sums are sums without overlap. The zero-representation condition is often the easiest way to test uniqueness.

Key Insight: A sum is direct precisely when there is no nontrivial way for the pieces to cancel each other.

Remark 2.1.24. *Products and Direct Sums*
For finitely many vector spaces $\mathcal{V}_1, \dots, \mathcal{V}_m$, the cartesian product

$$\mathcal{V}_1 \times \cdots \times \mathcal{V}_m$$

with componentwise operations is again a vector space. In the finite setting this is also the external direct sum, so products and direct sums coincide as vector spaces. This viewpoint is often the cleanest way to understand block decompositions later on.

Theorem 2.1.25. *Dimension Formula for a Sum* *If $\mathcal{U}, \mathcal{W}$ are finite-dimensional subspaces of $\mathcal{V}$, then*

$$\dim(\mathcal{U} + \mathcal{W}) = \dim(\mathcal{U}) + \dim(\mathcal{W}) - \dim(\mathcal{U} \cap \mathcal{W}).$$

Proof. Choose a basis $(\mathbf{e}_1, \dots, \mathbf{e}_r)$ of $\mathcal{U} \cap \mathcal{W}$, extend it to a basis

$$(\mathbf{e}_1, \dots, \mathbf{e}_r, \mathbf{u}_1, \dots, \mathbf{u}_p)$$

of $\mathcal{U}$, and to a basis

$$(\mathbf{e}_1, \dots, \mathbf{e}_r, \mathbf{w}_1, \dots, \mathbf{w}_q)$$

of $\mathcal{W}$. Then

$$(\mathbf{e}_1, \dots, \mathbf{e}_r, \mathbf{u}_1, \dots, \mathbf{u}_p, \mathbf{w}_1, \dots, \mathbf{w}_q)$$

is a basis of $\mathcal{U} + \mathcal{W}$, giving

$$\dim(\mathcal{U} + \mathcal{W}) = r + p + q = (r + p) + (r + q) - r.$$

Structural Commentary: The formula corrects double-counting: vectors in $\mathcal{U} \cap \mathcal{W}$ are counted once in each subspace. □

Key Insight: The dimension of a sum is the sum of dimensions minus the overlap.

Corollary 2.1.26. *If $\mathcal{U} \cap \mathcal{W} = \{\mathbf{0}\}$, then*

$$\dim(\mathcal{U} \oplus \mathcal{W}) = \dim(\mathcal{U}) + \dim(\mathcal{W}).$$

Structural Commentary: When the overlap is trivial, there is no double-counting in the dimension formula.

Key Insight: Dimensions add exactly for direct sums.

Proposition 2.1.27. *Complements Exist in Finite Dimensions*
If $\mathcal{V}$ is finite-dimensional and $\mathcal{U} \subseteq \mathcal{V}$ is a subspace, then there exists a subspace $\mathcal{W}$ such that

$$\mathcal{V} = \mathcal{U} \oplus \mathcal{W}.$$

Proof. Choose a basis of $\mathcal{U}$ and extend it to a basis of $\mathcal{V}$. Let $\mathcal{W}$ be the span of the added basis vectors. *Structural Commentary*: Complements split a finite-dimensional space into a chosen subspace and the remaining directions. □

Key Insight: Every finite-dimensional subspace can be paired with another subspace to recover the whole space as a direct sum.

Definition 2.1.28. *Quotient Space*
Let $\mathcal{U}$ be a subspace of $\mathcal{V}$. For $\mathbf{v} \in \mathcal{V}$, the coset of $\mathbf{v}$ modulo $\mathcal{U}$ is

$$\mathbf{v} + \mathcal{U} := \{\mathbf{v} + \mathbf{u} : \mathbf{u} \in \mathcal{U}\}.$$

The quotient space $\mathcal{V}/\mathcal{U}$ is the set of all cosets $\mathbf{v} + \mathcal{U}$.

Proposition 2.1.29. *Structure of the Quotient*
Addition and scalar multiplication on $\mathcal{V}/\mathcal{U}$ defined by

$$(\mathbf{v} + \mathcal{U}) + (\mathbf{w} + \mathcal{U}) := (\mathbf{v} + \mathbf{w}) + \mathcal{U}, \qquad a(\mathbf{v} + \mathcal{U}) := a\mathbf{v} + \mathcal{U}$$

are well-defined, and with these operations $\mathcal{V}/\mathcal{U}$ is a vector space.

Proof. If $\mathbf{v}+\mathcal{U}=\mathbf{v}'+\mathcal{U}$ and $\mathbf{w}+\mathcal{U}=\mathbf{w}'+\mathcal{U}$, then $\mathbf{v}-\mathbf{v}'\in\mathcal{U}$ and $\mathbf{w}-\mathbf{w}'\in\mathcal{U}$. Because $\mathcal{U}$ is a subspace,

$$(\mathbf{v}+\mathbf{w})-(\mathbf{v}'+\mathbf{w}')\in\mathcal{U} \quad \text{and} \quad a\mathbf{v}-a\mathbf{v}'\in\mathcal{U},$$

so the operations do not depend on the chosen representatives. The vector space axioms then follow from the corresponding axioms in $\mathcal{V}$. *Structural Commentary*: Quotients inherit linear structure precisely because changing representatives only changes vectors by elements of $\mathcal{U}$. □

Key Insight: A quotient space treats vectors differing by an element of $\mathcal{U}$ as the same vector.

Definition 2.1.30. *Quotient Map*
The map

$$\pi:\mathcal{V}\to\mathcal{V}/\mathcal{U}, \qquad \pi(\mathbf{v})=\mathbf{v}+\mathcal{U},$$

is called the quotient map.

Proposition 2.1.31. *The quotient map π is linear, surjective, and*

$$\ker(\pi)=\mathcal{U}.$$

Structural Commentary: The quotient map is the canonical linear map that collapses $\mathcal{U}$ to zero and keeps track of cosets.

Key Insight: Quotient maps turn subspaces into kernels.

Theorem 2.1.32. *Dimension of a Quotient*
If $\mathcal{V}$ is finite-dimensional and $\mathcal{U}$ is a subspace, then

$$\dim(\mathcal{V}/\mathcal{U})=\dim(\mathcal{V})-\dim(\mathcal{U}).$$

Proof. Choose a basis

$$(\mathbf{u}_1,\dots,\mathbf{u}_m,\mathbf{w}_1,\dots,\mathbf{w}_k)$$

of $\mathcal{V}$ such that $(\mathbf{u}_1,\dots,\mathbf{u}_m)$ is a basis of $\mathcal{U}$. Then every coset in $\mathcal{V}/\mathcal{U}$ has a unique form

$$a_1(\mathbf{w}_1+\mathcal{U})+\cdots+a_k(\mathbf{w}_k+\mathcal{U}),$$

so $(\mathbf{w}_1+\mathcal{U},\dots,\mathbf{w}_k+\mathcal{U})$ is a basis of $\mathcal{V}/\mathcal{U}$. Hence

$$\dim(\mathcal{V}/\mathcal{U})=k=\dim(\mathcal{V})-\dim(\mathcal{U}).$$

Structural Commentary: The quotient removes the directions belonging to $\mathcal{U}$ and retains the complementary degrees of freedom. □

Key Insight: Passing to $\mathcal{V}/\mathcal{U}$ subtracts the dimension of the collapsed subspace.

Remark 2.1.33. *Structural Viewpoint*
Passing to a quotient means declaring every vector in $\mathcal{U}$ to be invisible. Direct sums split a space into independent pieces; quotients collapse a chosen piece to zero. These are complementary ways of extracting structure, and they reappear constantly in the study of linear maps through kernels, ranges, and induced maps.

2.1.2 Linear Maps and Representations

This subsection blends the two standard viewpoints. Axler emphasises linear maps as the primary objects and matrices as secondary coordinate descriptions; Garcia and Horn emphasises how matrices expose structure. We will use both viewpoints together.

Definition 2.1.34. *Linear Map*
Let $\mathcal{V}$ and $\mathcal{W}$ be vector spaces over the same field $\mathbb{F}$. A function

$$T:\mathcal{V}\to\mathcal{W}$$

is linear if for all $\mathbf{u},\mathbf{v}\in\mathcal{V}$ and all $a,b\in\mathbb{F}$,

$$T(a\mathbf{u}+b\mathbf{v})=aT(\mathbf{u})+bT(\mathbf{v}).$$

The set of all linear maps from $\mathcal{V}$ to $\mathcal{W}$ is denoted $\mathcal{L}(\mathcal{V},\mathcal{W})$. If $\mathcal{V}=\mathcal{W}$, we write $\mathcal{L}(\mathcal{V})$ and call such a map an operator.

Remark 2.1.35. *Coordinate-Free Principle*
A linear map is completely determined by what it does to a basis. Coordinates do not create the map; they only encode it once bases have been chosen.

Proposition 2.1.36. *If $(\mathbf{v}_1,\dots,\mathbf{v}_n)$ is a basis of $\mathcal{V}$ and $\mathbf{w}_1,\dots,\mathbf{w}_n\in\mathcal{W}$ are arbitrary, then there exists a unique linear map $T:\mathcal{V}\to\mathcal{W}$ such that*

$$T(\mathbf{v}_j)=\mathbf{w}_j \qquad \text{for } j=1,\dots,n.$$

Proof. Every $\mathbf{v}\in\mathcal{V}$ has a unique expression $\mathbf{v}=a_1\mathbf{v}_1+\cdots+a_n\mathbf{v}_n$. Define

$$T(\mathbf{v}) := a_1\mathbf{w}_1+\cdots+a_n\mathbf{w}_n.$$

This is well-defined because basis coordinates are unique, and linearity is immediate. □

Definition 2.1.37. *Kernel and Range*
For $T\in\mathcal{L}(\mathcal{V},\mathcal{W})$, define

$$\ker(T) := \{\mathbf{v}\in\mathcal{V} : T(\mathbf{v})=\mathbf{0}\}, \qquad \operatorname{range}(T) := \{T(\mathbf{v}) : \mathbf{v}\in\mathcal{V}\}.$$

Proposition 2.1.38. *If $T\in\mathcal{L}(\mathcal{V},\mathcal{W})$, then $\ker(T)$ is a subspace of $\mathcal{V}$ and $\operatorname{range}(T)$ is a subspace of $\mathcal{W}$.*

Proposition 2.1.39. *A linear map $T\in\mathcal{L}(\mathcal{V},\mathcal{W})$ is injective if and only if*

$$\ker(T)=\{\mathbf{0}\}.$$

Theorem 2.1.40. *Rank-Nullity Theorem*
If $\mathcal{V}$ is finite-dimensional and $T\in\mathcal{L}(\mathcal{V},\mathcal{W})$, then

$$\dim(\mathcal{V})=\dim(\ker(T))+\dim(\operatorname{range}(T)).$$

Proof. Let $(\mathbf{u}_1,\dots,\mathbf{u}_m)$ be a basis of $\ker(T)$ and extend it to a basis

$$(\mathbf{u}_1,\dots,\mathbf{u}_m,\mathbf{w}_1,\dots,\mathbf{w}_r)$$

of $\mathcal{V}$. We claim that

$$(T\mathbf{w}_1,\dots,T\mathbf{w}_r)$$

is a basis of $\operatorname{range}(T)$. It spans because every vector in $\mathcal{V}$ decomposes into a kernel part plus a combination of the $\mathbf{w}_j$, and it is linearly independent because a relation

$$a_1T\mathbf{w}_1+\cdots+a_rT\mathbf{w}_r=\mathbf{0}$$

would imply $a_1\mathbf{w}_1+\cdots+a_r\mathbf{w}_r\in\ker(T)$, contradicting linear independence of the basis of $\mathcal{V}$. Therefore

$$\dim(\operatorname{range}(T))=r, \qquad \dim(\ker(T))=m,$$

and $\dim(\mathcal{V})=m+r$. □

Definition 2.1.41. *Rank and Nullity*
For a finite-dimensional domain, the rank of T is $\dim(\operatorname{range}(T))$ and the nullity of T is $\dim(\ker(T))$.

Definition 2.1.42. *Invertible Linear Map and Isomorphism*
A linear map $T\in\mathcal{L}(\mathcal{V},\mathcal{W})$ is invertible if there exists $S\in\mathcal{L}(\mathcal{W},\mathcal{V})$ such that

$$ST=I_{\mathcal{V}} \qquad \text{and} \qquad TS=I_{\mathcal{W}}.$$

An invertible linear map is called an isomorphism, and if one exists we write $\mathcal{V}\cong\mathcal{W}$.

Proposition 2.1.43. *A linear map is invertible if and only if it is both injective and surjective.*

Theorem 2.1.44. *If $\mathcal{V}$ is finite-dimensional and $T\in\mathcal{L}(\mathcal{V})$, then the following are equivalent:*

i. T is invertible;

ii. T *is injective;*

iii. T *is surjective.*

Proof. The implication $(i) \Rightarrow (ii)$ is immediate. If T is injective, then $\ker(T) = \{\mathbf{0}\}$, so rank-nullity gives

$$\dim(\text{range}(T)) = \dim(\mathcal{V}),$$

hence $\text{range}(T) = \mathcal{V}$ and T is surjective. The implication $(iii) \Rightarrow (i)$ follows similarly. □

Theorem 2.1.45. *Two finite-dimensional vector spaces are isomorphic if and only if they have the same dimension.*

Definition 2.1.46. *Matrix Representation*
Let $\beta = (\mathbf{v}_1, \dots, \mathbf{v}_n)$ be a basis of $\mathcal{V}$ and $\gamma = (\mathbf{w}_1, \dots, \mathbf{w}_m)$ a basis of $\mathcal{W}$. For $\mathbf{v} \in \mathcal{V}$, denote its coordinate column vector by $[\mathbf{v}]_\beta$. For $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, the matrix of T relative to (β, γ) is the unique matrix $[T]_{\gamma \leftarrow \beta} \in \text{Mat}(m, n, \mathbb{F})$ whose jth column is $[T(\mathbf{v}_j)]_\gamma$.

Proposition 2.1.47. *For every* $\mathbf{v} \in \mathcal{V}$,

$$[T(\mathbf{v})]_\gamma = [T]_{\gamma \leftarrow \beta}[\mathbf{v}]_\beta.$$

Proof. Write $\mathbf{v} = a_1\mathbf{v}_1 + \cdots + a_n\mathbf{v}_n$. Then linearity gives

$$T(\mathbf{v}) = a_1T(\mathbf{v}_1) + \cdots + a_nT(\mathbf{v}_n),$$

and the right-hand side is exactly the linear combination of the columns of $[T]_{\gamma \leftarrow \beta}$ with coefficients $a_1, \dots, a_n$. □

Proposition 2.1.48. *Composition and Matrix Multiplication*
If $T \in \mathcal{L}(\mathcal{U}, \mathcal{V})$ *and* $S \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, *then*

$$[ST]_{\delta \leftarrow \alpha} = [S]_{\delta \leftarrow \beta}[T]_{\beta \leftarrow \alpha}$$

for any bases α *of* $\mathcal{U}$, β *of* $\mathcal{V}$, *and* δ *of* $\mathcal{W}$.

Theorem 2.1.49. *Linear Maps and Matrices Are the Same Data*
If $\dim(\mathcal{V}) = n$ *and* $\dim(\mathcal{W}) = m$, *then choosing bases of* $\mathcal{V}$ *and* $\mathcal{W}$ *gives a vector-space isomorphism*

$$\mathcal{L}(\mathcal{V}, \mathcal{W}) \cong \text{Mat}(m, n, \mathbb{F}).$$

Consequently,

$$\dim(\mathcal{L}(\mathcal{V}, \mathcal{W})) = mn.$$

Remark 2.1.50. *Why This Matters*
This is the key bridge between abstraction and computation. The linear map is basis-free; the matrix is what the map looks like after coordinates have been chosen.

Definition 2.1.51. *Change of Basis Matrix*
Let β and γ be two bases of the same vector space $\mathcal{V}$. The change-of-basis matrix from β-coordinates to γ-coordinates is

$$C_{\gamma \leftarrow \beta} := [I_\mathcal{V}]_{\gamma \leftarrow \beta}.$$

Thus

$$[\mathbf{v}]_\gamma = C_{\gamma \leftarrow \beta}[\mathbf{v}]_\beta$$

for every $\mathbf{v} \in \mathcal{V}$.

Proposition 2.1.52. *The change-of-basis matrix* $C_{\gamma \leftarrow \beta}$ *is invertible, with inverse*

$$C_{\beta \leftarrow \gamma}.$$

Theorem 2.1.53. *Change of Basis Formula*
Let $T \in \mathcal{L}(\mathcal{V})$, *and let* β, γ *be two bases of* $\mathcal{V}$. *Then*

$$[T]_{\gamma \leftarrow \gamma} = C_{\gamma \leftarrow \beta}[T]_{\beta \leftarrow \beta}C_{\beta \leftarrow \gamma}.$$

Proof. Use the identity

$$[T(\mathbf{v})]_\gamma = C_{\gamma\leftarrow\beta}[T(\mathbf{v})]_\beta = C_{\gamma\leftarrow\beta}[T]_{\beta\leftarrow\beta}[\mathbf{v}]_\beta = C_{\gamma\leftarrow\beta}[T]_{\beta\leftarrow\beta}C_{\beta\leftarrow\gamma}[\mathbf{v}]_\gamma.$$

for all $\mathbf{v} \in \mathcal{V}$. □

Definition 2.1.54. *Similarity*
Matrices $A, B \in \mathrm{Mat}(n, n, \mathbb{F})$ are similar if there exists an invertible matrix P such that

$$B = PAP^{-1}.$$

Remark 2.1.55. *Similarity as Same Operator, New Coordinates*
Similarity does not mean two operators are merely related; it means they are the same operator written in two different bases. This is the structural content behind change of basis.

Definition 2.1.56. *Block Matrices*
Suppose

$$\mathcal{V} = \mathcal{U}_1 \oplus \mathcal{U}_2 \qquad \text{and} \qquad \mathcal{W} = \mathcal{Z}_1 \oplus \mathcal{Z}_2.$$

If $T \in \mathcal{L}(\mathcal{V}, \mathcal{W})$, then relative to bases adapted to these decompositions, the matrix of T has block form

$$[T] = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

where each block records the part of T mapping one summand of the domain into one summand of the codomain.

Proposition 2.1.57. *Invariant Subspaces and Block Form*
Let $T \in \mathcal{L}(\mathcal{V})$ and suppose

$$\mathcal{V} = \mathcal{U} \oplus \mathcal{W}.$$

i. If $\mathcal{U}$ is invariant under T, then in a basis adapted to $\mathcal{U} \oplus \mathcal{W}$ the matrix of T has the upper block-triangular form

$$\begin{bmatrix} A & B \\ 0 & D \end{bmatrix}.$$

ii. If both $\mathcal{U}$ and $\mathcal{W}$ are invariant under T, then the matrix of T has the block-diagonal form

$$\begin{bmatrix} A & 0 \\ 0 & D \end{bmatrix}.$$

Proof. If $\mathcal{U}$ is invariant, then $T(\mathcal{U}) \subseteq \mathcal{U}$, so no basis vector from $\mathcal{U}$ can acquire a component in $\mathcal{W}$. That forces the lower-left block to be zero. If $\mathcal{W}$ is also invariant, the same argument forces the upper-right block to be zero for the complementary direction as well. □

Remark 2.1.58. *Matrix Viewpoint on Structure*
Basic block matrices already capture important geometry. Upper block-triangular form records the presence of an invariant subspace; block diagonal form records a genuine decomposition into independent invariant pieces.

3 Time Series Analysis

3.1 Time Series Analysis

Based on the books by James Douglas **?**, and ...

3.1.1 Stationary Time Series

Definition 3.1.1. A *linear first-order difference equation* is defined as

$$y_t = \phi y_{t-1} + w_t$$

where y_t is the target variable (with y_{t-1} the lag 1 variable), w_t is an input variable at time t

The difference equation may be solved by recursive substitution to arrive at

$$y_t = \phi^{t+1} y_{-1} + \phi^t w_0 + \phi^{t-1} w_1 + \phi^{t-2} w_2 + \cdots + \phi w_{t-1} + w_t$$

The *dynamic multiplier* calculates effect of w_t on y_{t+j}, and is given by

$$\frac{\partial y_{t+j}}{\partial w_t} = \phi^j$$

If $|\phi| < 1$, the system is stable. If $|\phi| > 1$, then the system is explosive.

We may generalise the process to *p-th order difference equation*, i.e.,

$$y_t = \phi_t y_{t-1} + \phi_2 t_{t-2} + \cdots + \phi_p y_{t-p} + w_t$$

We may rewrite this as a first-order difference equation in a $(p \times 1)$ vector $\boldsymbol{\xi}_t$:

$$\boldsymbol{\xi}_t = \begin{bmatrix} y_t \\ y_{t-1} \\ \vdots \\ y_{t-p+1} \end{bmatrix}$$

Define the $(p \times p)$ matrix $\boldsymbol{F}$ by

$$\boldsymbol{F} = \begin{bmatrix} \phi_1 & \phi_2 & \phi_3 & \cdots & \phi_{p-1} & \phi_p \\ 1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & 0 \end{bmatrix}$$

Finally, define the $(p \times 1)$ vector $\boldsymbol{v}_t$ by

$$\boldsymbol{v}_t = \begin{bmatrix} w_t \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Then the system of p equations $\boldsymbol{\xi}_t = \boldsymbol{F}\boldsymbol{\xi}_{t-1} + \boldsymbol{v}_t$ is identical to the first order difference equation, and by recursive substitution, we have the following for the case of p-th order difference equation:

$$\boldsymbol{\xi}_{t+j} = \boldsymbol{F}^{j+1}\boldsymbol{\xi}_{t-1} + \boldsymbol{F}^j \boldsymbol{v}_t + \boldsymbol{F}^{j-1}\boldsymbol{v}_{t+1} + \cdots + \boldsymbol{F}\boldsymbol{v}_{t+j-1} + \boldsymbol{v}_{t+j}$$

Hence the dynamic multiplier is then $\frac{\partial y_{t+j}}{\partial w_t} f_{11}^{(j)}$, where $f_{11}^{(j)}$ is the $(1,1)$ element of $\boldsymbol{F}^j$.

3.1.2 Univariate Time Series Models

Trading activities through an exchange can be described by a sequence of time stamps ('ticks') $t_0 < t_1 < \cdots < t_n$, and 'marks' y_i at time t_i, where t_i denote market open, t_n denote market close. The marks y_i is a characteristic of the order book at time of ith activity. Events with marks associated with the ticks can be described mathematically as a marked point process.

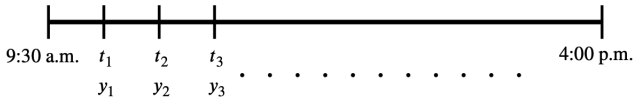


Figure 1: Ticks and Marks

A data aggregation method is where aggregation is conducted when there is a change in marker. An alternative aggregation method would be to divide the time span T for exchange hours into K intervals, so regularly spaced intervals are of size $\Delta t = T/K$. The time series method to be discussed will apply to all aggregated data.

Let $p_{it} = \ln P_{it}$ denote price of ith asset at time t; $p_t = (p_{1t}, p_{2t}, \ldots, p_{nt})$ denote price vector for n assets; y_{it} denote vector of characteristics of ith asset at time t. These quantities are aggregated from high frequency data. Consider r factors $f_t = (f_{1t}, f_{2t}, \ldots, f_{rt})$ that may include market and industry factors, and asset characteristics. Trading rules can be broadly grouped as follows:

i. *Statistical Arbitrage*: $E(p_{i,t+1} \mid p_{i,t}, p_{i,t-1}, \ldots, y_{i,t}, y_{i,t-1}, \ldots)$, that predicts the price of ith stock at $t+1$ based on past trading information (also known as time series momentum)

ii. *Momentum*: $E(p_{t+1} \mid p_t, p_{t-1}, \ldots, y_t, y_{t-1}, \ldots)$, that predicts the cross-sectional momentum of a subset of stocks based on past trading characteristics. For portfolio formation and rebalancing, pairs trading

iii. *Fair Value*: $E(p_{t+1} \mid p_t, p_{t-1}, \ldots, y_t, y_{t-1}, \ldots, f_t, f_{t-1}, \ldots)$, predicts price using all relevant quantities. Factors normally include market, Fama-French; at a more macro level than timescale considered for price prediction, but may still be useful.

Hence price and volatility prediction can be formulated as a time series prediction problem. Autocorrelations and partial autocorrelations can be used to build autoregressive and ARCH models with some predictive power.

Let $Y_1, Y_2, \ldots, Y_T$ be a sequence of random variables with a joint probability distribution. A sequence of observations of stochastic process $\{Y_t, t = 1, \ldots, T\}$ is a realisation of the process.
A time series $\{Y_t\}$ is *stationary* if for every integer m, the set of variables $Y_{t_1}, Y_{t_2}, \ldots, Y_{t_m}$ depends only on the distance between times $t_1, t_2, \ldots, t_m$. Thus $E(Y_t) = \mu$, $Var(Y_t) = \sigma^2$ are constant for all t.
The *auto-covariance* function is defined as

$$\gamma(s) = Cov(Y_t, Y_{t-s}) = E[(Y_t - \mu)(Y_{t-s} - \mu)] \quad \forall s = 0, \pm 1, \ldots$$

The *auto-correlation* of process at lag s is defined as

$$\rho(s) = Corr(Y_t, Y_{t-s}) = \frac{Cov(Y_t, Y_{t-s})}{(Var(Y_t)\ Var(Y_{t-s}))^{1/2}} = \frac{\gamma(s)}{\gamma(0)}, \quad \forall s = 0, \pm 1, \ldots$$

Some examples of stationary stochastic processes are as follows:

Example 3.1.2. *(White Noise)* Sequence of discrete independent random variables (r.v.) $\{\epsilon_t\}$ with $E[\epsilon_t] = 0$ and $E[\epsilon_t^2] = \sigma^2$. Set $Y_t = \mu + \epsilon_t$, then $E[Y_t] = \mu$, $Cov(Y_t, Y_{t-s}) = Var(Y_t) = \sigma^2$ if $s = 0$, and $Cov(Y_t, Y_{t-s}) = 0$ if $s \neq 0$. Hence the process is stationary.

Example 3.1.3. *(Moving Average)* Let $\{\epsilon_t\}$ be independent r.v., with process $\{Y_t\}$ where $Y_t = \mu + \epsilon_t + \epsilon_{t_1}$ for $t = 0, 1, 2, \ldots$, and μ is constant. Then $E[Y_t] = \mu\ \forall t$, and

$$Cov(Y_t, Y_{t-s}) = \gamma(s) = \begin{cases} 2\sigma^2 & \text{if } s = 0 \\ \sigma^2 & \text{if } s = 1 \\ 0 & \text{if } s > 1 \end{cases}$$

Hence $\{Y_t\}$ is stationary, with $\rho(s) = \gamma(s)/\gamma(0)$ such that $\rho(0) = 1$, $\rho(1) = 1/2$, $\rho(s) = 0$ for $|s| > 1$.

Some examples of non-stationary stochastic processes are as follows:

Example 3.1.4. *(Random Walk with Drift)* Let $\{\epsilon_t\}$ be sequence of independent r.v. with $E[\epsilon_t] = 0$, $E[\epsilon_t^2] = \sigma^2$, and define process $\{Y_t\}$ by $Y_t = Y_{t-1} + \delta + \epsilon_t$ with $Y_0 = 0$. Then the process can be summarised as

$$Y_t = \delta t + \sum_{j=1}^{t} \epsilon_j$$

Note that $E[Y_t] = \delta t$ and $Var[Y_t] = t\sigma^2$, hence process $\{Y_t\}$ is not stationary.

Given T observations from stationary process $\{Y_t\}$, the *sample mean* is $\overline{Y} = \frac{1}{T}\sum_{t=1}^{T} Y_t$. The *sample auto-covariance* function is defined by $\hat{\gamma}(s) = \frac{1}{T}\sum_{t=1}^{T-s}(Y_t - \overline{Y})(Y_{T-s} - \overline{Y})$ for $s = 0, 1, \ldots$. The *sample auto-correlation* function (ACF) is defined as $\hat{\rho}(s) = \frac{\hat{\gamma}(s)}{\hat{\gamma}(0)} = r(s)$. The *sample variance* is $Var(\overline{Y}) = \frac{\gamma(0)}{T}\left[1 + 2\sum_{u=1}^{Y-1}\left(\frac{T-u}{T}\right)\rho(u)\right]$; note that it has to account for auto-correlations.

Definition 3.1.5. A stochastic process $\{Y_t\}$ is a *linear process* if it can be represented as

$$Y_t = \mu + \sum_{j=0}^{\infty} \Psi_j \epsilon_{t-j}$$

where ϵ_t are independent with mean 0, variance σ_ϵ^2, and $\sum_{j=0}^{\infty} < \infty$.

4 Mathematical and Statistical Foundation

4.1 Classical Machine Learning

4.1.1 Ensemble Methods

4.2 Deep Learning

4.2.1 Deep Feedforward Networks

Remark 4.2.1. *Deep Feedforward Networks (DFNs)*
Defines a mapping $\boldsymbol{y} = f(\boldsymbol{x}; \boldsymbol{\theta})$ and learns value of parameters $\boldsymbol{\theta}$ that results in best function approximation.

Remark 4.2.2. *Linear Models*
Linear models may fit data efficiently and reliably, either in closed form or with convex optimisation.
Model capacity limited to linear functions, does not model interaction between any two input variables

Remark 4.2.3. *Nonlinear Models*
To represent nonlinear functions of $\boldsymbol{x}$, apply linear model to transformed input $\phi(\boldsymbol{x})$ where ϕ is a nonlinear transformation. Kernel trick may be applied to obtain nonlinear learning algorithm. To choose mapping ϕ:

i. Choose generic ϕ, such as that used by kernel machines based on RBF kernel.
If $\phi(\boldsymbol{x})$ is of high enough dimension, can find enough capacity to fit training set, but generalisations to test set remains poor. Mappings are based on principle of local smoothness and do not encode enough prior information to solve advanced problems.

ii. Manually engineer ϕ. Requires decades of human effort for each task, and practitioners specialising in different domains (i.e, speech recognition, computer vision) with little transfer between domains.

iii. Learn ϕ through deep learning. Model is $y = f(\boldsymbol{x}; \boldsymbol{\theta}, \boldsymbol{w}) = \phi(\boldsymbol{x}; \boldsymbol{\theta})^T \boldsymbol{w}$, where parameters $\boldsymbol{\theta}$ can be used to learn ϕ from broad class of functions, and parameters $\boldsymbol{w}$ that map from $\phi(\boldsymbol{x})$ to desired output.
Do not require training problem to be convex, and only require finding the right general function.

Definition 4.2.4. *Cost Functions*

i. Learning Conditional Distributions with Maximum Likelihood: cost function is negative log-likelihood, which is the cross-entropy between training data and model distribution:

$$J(\boldsymbol{\theta}) = -\mathbb{E}_{\boldsymbol{x},\boldsymbol{y}\sim\hat{p}_{\text{data}}} \log p_{\text{model}}(\boldsymbol{y} \mid \boldsymbol{x})$$

Specific form of cost function changes from model to model, depending on form of $\log p_{\text{model}}$.
Method removes the burden of designing cost functions for each model, as specifying a model $p(\boldsymbol{y} \mid \boldsymbol{x})$ automatically determines a cost function $\log p(\boldsymbol{y} \mid \boldsymbol{x})$.
Gradient of cost function for neural networks must be large and predictable enough to serve as good guide for the learning algorithm. The negative log-likelihood helps to avoid saturation of the function.

ii. Learning Conditional Statistics: to learn just one conditional statistic of $\boldsymbol{y}$ given $\boldsymbol{x}$.
With sufficiently powerful neural network, this can represent any function f from a wide class of function, limited by features of continuity and boundedness. Hence the cost function is a functional. Learning is choosing a functional rather than a set of parameters.
By calculus of variation , solving the optimisation problem yields the below function,

$$f^* = \arg\min_f \mathbb{E}_{\boldsymbol{x},\boldsymbol{y}\sim p_{\text{data}}} \|\boldsymbol{y} - f(\boldsymbol{x})\|^2$$
$$f^* = \mathbb{E}_{\boldsymbol{y}\sim p_{\text{data}}(\boldsymbol{y} \mid \boldsymbol{x})}[\boldsymbol{y}]$$

If infinitely many samples from the true data-generating distribution could be trained, then minimising the mean squared error cost function gives a function that predicts mean of $\boldsymbol{y}$ for each value of $\boldsymbol{x}$.
A second result from calculus of variations is:

$$f^* = \arg\min_f \mathbb{E}_{\boldsymbol{x},\boldsymbol{y}\sim p_{\text{data}}} \|\boldsymbol{y} - f(\boldsymbol{x})\|_1$$

which is a function that predicts the median value of $\boldsymbol{y}$ for each $\boldsymbol{x}$. This is the mean absolute error.
Note that mean squared error and mean absolute error often lead to poor results when used with gradient-based optimisation. Output units that saturate may produce very small gradients when combined with these cost functions. Hence the reason that cross-entropy cost function is more popular.

Definition 4.2.5. *Output Units*

i. Linear Units: base on affine transformation with no nonlinearity.
Given features $\boldsymbol{h}$, a layer of output units produces vector $\hat{\boldsymbol{y}} = \boldsymbol{W}^T \boldsymbol{h} + \boldsymbol{b}$. Linear output layers are used to produce mean of conditional Gaussian distribution:

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{y}; \hat{\boldsymbol{y}}, \boldsymbol{I})$$

Maximising log-likelihood is equivalent to minimising mean squared error.
Linear units do not saturate, hence may be used for wide variety of optimisation algorithms.

ii. Sigmoid Units: define Bernoulli distribution y conditioned on $\boldsymbol{x}$. Neural net to predict $P(y = 1 \mid \boldsymbol{x})$, which lies in interval $[0, 1]$. The sigmoid unit is defined by

$$\hat{y} = \sigma(\boldsymbol{w}^T \boldsymbol{h} + b)$$

where σ is the logistic sigmoid function.
Note that the cost function used with maximum likelihood is $-\log P(y \mid \boldsymbol{x})$, preventing saturation.
The loss function for MLE of Bernoulli parametrised by sigmoid is

$$J(\boldsymbol{\theta}) = -\log P(y \mid \boldsymbol{x}) = -\log \sigma((2y-1)z) = \zeta((1-2y)z)$$

Function saturates only when $(1-2y)z$ is very negative, i.e., when model has the right answer.

iii. Softmax Units: used to represent probability distribution over n different classes.
A linear layer predicts unnormalised log probabilities:

$$\boldsymbol{z} = \boldsymbol{W}^T \boldsymbol{h} + \boldsymbol{b}$$

where $z_i = \log \tilde{P}(y = i \mid \boldsymbol{x})$. Softmax function is then

$$\text{softmax}(\boldsymbol{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

The function is to maximise $\log P(y = i; \boldsymbol{z}) = \log \text{softmax}(\boldsymbol{z})_i = z_i - \log \sum_j \exp(z_j)$.
Note that the input z_i always has direct contribution to cost function, and the term cannot saturate.
Un-regularised maximum likelihood will drive the model to learn parameters that drive the softmax to predict fraction of counts for each outcome observed in the training set:

$$\text{softmax}(z(\boldsymbol{x}; \boldsymbol{\theta}))_i \approx \frac{\sum_{j=1}^{m} \mathbf{1}_{y^{(j)}=i, \boldsymbol{x}^{(j)}=x}}{\sum_{j=1}^{m} \mathbf{1}_{\boldsymbol{x}^{(j)}=x}}$$

Note that objective functions other than log-likelihood does not work well with softmax function. Squared error is poor loss function for softmax units, and can fail to train the model to change its output.
Softmax function can saturate, and many functions based on softmax also saturate, unless they are able to invert the saturating activating function.

iv. Other Output Types: generally, given a conditional distribution $p(\boldsymbol{y} \mid \boldsymbol{x};\ \boldsymbol{\theta})$, principle of maximum likelihood suggests using $-\log p(\boldsymbol{y} \mid \boldsymbol{x};\ \boldsymbol{\theta})$ as the cost function.
Neural networks represent a function $f(\boldsymbol{x}; \boldsymbol{\theta}) = \boldsymbol{\omega}$. The outputs of the function are not direct predictions of value $\boldsymbol{y}$, but the parameters for distribution over $\boldsymbol{y}$. The loss function is then $-\log p(\boldsymbol{y}; \boldsymbol{\omega}(\boldsymbol{x}))$.

Remark 4.2.6. *Learning Distribution Parameters*

i. Heteroscedastic Model: to predict different variance in $\boldsymbol{y}$ for different values of $\boldsymbol{x}$, formulate the Gaussian distribution using precision rather than variance. In multi-variate case, the diagonal precision matrix is used, $\text{diag}(\boldsymbol{\beta})$. The log-likelihood of Gaussian distribution parametrised by $\boldsymbol{\beta}$ involves only multiplication by β_i and addition of $\log \boldsymbol{\beta}_i$. The gradient of these operations are well-behaved.
Let $\boldsymbol{\alpha}$ be raw activation of the model used to determine diagonal precision. The softplus function may be used to obtain a positive precision vector $\boldsymbol{\beta} = \zeta(\boldsymbol{\alpha})$. Same strategy applies equally if using variance variance or standard deviation rather than precision.
If covariance is full and conditional, then parametrisation must be chosen that guarantees positive-definiteness of predicted covariance matrix.

$$\boldsymbol{\Sigma}(\boldsymbol{x}) = \boldsymbol{B}(\boldsymbol{x})\boldsymbol{B}^T(\boldsymbol{x})$$

where $\boldsymbol{B}$ is unconstrained square matrix. Note that if matrix is full rank, then computing likelihood requires $O(d^3)$ a $d \times d$ matrix for the determinant and inverse of $\boldsymbol{\Sigma}(\boldsymbol{x})$.

ii. Mixture Density Networks: to perform multimodal regression (predict real values that come from condi-

tional distribution $p(\boldsymbol{y} \mid \boldsymbol{x})$) that can have several different peaks in $\boldsymbol{y}$ for the same $\boldsymbol{x}$.

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \sum_{i=1}^{n} p(c = i \mid \boldsymbol{x}) \mathcal{N}(\boldsymbol{y}; \boldsymbol{\mu}^{(i)}(\boldsymbol{x}), \boldsymbol{\Sigma}^{(i)}(\boldsymbol{x}))$$

The neural network will have three outputs:

1. Mixture components $p(c = i \mid \boldsymbol{x})$, forming a multinoulli distribution over n different components with latent variable c, obtained by softmax over n-dimensional vector.
2. Means $\boldsymbol{\mu}^{(i)}(\boldsymbol{x})$
3. Covariances $\boldsymbol{\Sigma}^{(i)}(\boldsymbol{x})$

Gradient-based optimisation of conditional Gaussian mixtures can be unreliable as the divisions can be numerically unstable. May be solved by clipping gradients, or scaling gradients heuristically.

Definition 4.2.7. *Hidden Units*
Even if hidden units are not differentiable at all input points, gradient descent still performs well enough as the training algorithms do not usually arrive at local minimum of cost function, but reduce its value significantly. Most hidden units has input vector $\boldsymbol{x}$, computes an affine transformation $\boldsymbol{z} = \boldsymbol{W}^T\boldsymbol{x} + \boldsymbol{b}$, then applying element-wise non-linear function $g(\boldsymbol{z})$.

i. Rectified Linear Units (ReLUs): uses activation function $g(z) = \max\{0, z\}$.
Typically used on top of an affine transformation $\boldsymbol{h} = g(\boldsymbol{W}^T\boldsymbol{x} + \boldsymbol{b})$. In initialisation, set all elements of $\boldsymbol{b}$ to small positive values, so that ReLUs will be initially active for most inputs in training set.
Generalisations of ReLUs have non-zero slope α_i for $z_i < 0$: $h_i = \max(0, z_i) + \alpha_i \min(0, z_i)$.

   1. Absolute Value Rectification: sets $\alpha_i = -1$ to obtain $g(z) = |z|$. Used for object recognition from images, to seek features that are invariant under polarity reversal of input illumination.
   2. Leaky ReLU: fixes α_i to small positive value
   3. Parametric ReU: treats α_i as a learnable parameter

ii. Maxout Units: divide $\boldsymbol{z}$ into groups of k values. Each maxout unit then outputs maximum element of one of these groups: $g(\boldsymbol{z})_i = \max_{j \in \mathbb{G}^{(i)}} z_j$, where $\mathbb{G}^{(i)}$ is indices of inputs for group i, which is $\{(i-1)k+1, \ldots, ik\}$.
This allows learning of piecewise linear function that responds to multiple directions in input $\boldsymbol{x}$ space .
Maxout units learn the activation function itself. With large k, maxout unit can learn to approximate any convex function. Each maxout unit is parametrised by k weight vectors, hence need more regularisation than ReLUs. Benefits include:

   1. Can work well without regularisation if training set is large and number of pieces per unit is low.
   2. Can gain statistical and computation advantages by requiring fewer parameters.
   3. Have redundancy that resists catastrophic forgetting, where neural networks forgot how to perform tasks that were trained on in the past.

iii. Logistic Sigmoid and Hyperbolic Tangent: the logistic sigmoid activation function is $g(z) = \sigma(z)$, and the hyperbolic tangent activation function is $g(z) = \tanh(z)$. Note that $\tanh(z) = 2\sigma(2z) - 1$.
Sigmoidal units saturate across most of the domain, which makes gradient-based learning very difficult. Hence the use in hidden units in feedforward networks is now discouraged.
If sigmoidal activation function must be used, hyperbolic tangent activation function performs better, as it resembles identity function more closely. Training $\hat{y} = \boldsymbol{w}^T \tanh(\boldsymbol{U}^T \tanh(\boldsymbol{V}^T\boldsymbol{x}))$ resembles training a linear model $\hat{y} = \boldsymbol{w}^T\boldsymbol{U}^T\boldsymbol{V}^T\boldsymbol{x}$ as long as the activations of the network can be kept small.
Sigmoidal functions are more common in recurrent networks, probabilities models, auto-encoders

iv. Linear Unit: consider neural network layer with n inputs, p outputs, $\boldsymbol{h} = g(\boldsymbol{W}^T\boldsymbol{x} + \boldsymbol{b})$. Replace with two layers, one using weight matrix $\boldsymbol{U}$ and the other using weight matrix $\boldsymbol{V}$. If first layer has no activation function, then the factored approach is to compute $\boldsymbol{h} = g(\boldsymbol{V}^T\boldsymbol{U}^T\boldsymbol{x} + \boldsymbol{b})$. If $\boldsymbol{U}$ produces q outputs, then $\boldsymbol{U}$ and $\boldsymbol{V}$ together only contains $(n + p)q$ parameters, while $\boldsymbol{W}$ contains np parameters. For small q, this is considerable saving in parameters, while the cost is constraining linear transformation to be low rank. This is an efficient way of reducing number of parameters in the model.

v. Softmax: naturally represent probability distribution over discrete variable with k possible values. Used only in more advanced architectures that explicitly learn to manipulate memory.

vi. Radial Basis Function (RBF): function becomes more active as $\boldsymbol{x}$ approaches a template $\boldsymbol{W}_{:,i}$. As it saturates to 0 for most $\boldsymbol{x}$, it can be difficult to optimise.

$$h_t = \exp\left(-\frac{1}{\sigma_i^2}\|\boldsymbol{W}_{:,i} - \boldsymbol{x}\|^2\right)$$

vii. Softplus: smooth version of rectifier for functional approximation and for conditional distributions of undirected probabilistic models. Usage is generally discouraged.

$$g(a) = \zeta(a) = \log(1 + e^a)$$

viii. Hard tanh: shaped similarly to tanh but bounded.

$$g(a) = \max(-1, \min(1, a))$$

Theorem 4.2.8. *Universal Approximation Theorem*
A feedforward network with linear output layer and at least one hidden layer with any 'squashing' activation function can approximate any Borel measurable function from one finite-dimensional space to another with any desired non-zero amount of error, provided the network is given enough hidden units.

Method 4.2.9. *Architecture Design*
Neural network layers are arranged in a chain structure, with each layer being a function of preceding layer.

$$\boldsymbol{h}^{(1)} = g^{(1)}(\boldsymbol{W}^{(1)T}\boldsymbol{x} + \boldsymbol{b}^{(1)})$$
$$\boldsymbol{h}^{(k)} = g^{(k)}(\boldsymbol{W}^{(k)T}\boldsymbol{h}^{(k-1)} + \boldsymbol{b}^{(k)}), \quad k \geq 2$$

The main considerations are the depth of network and width of each layer. Deeper networks use far fewer units per layers and far fewer parameters, and often generalise to the test set, but are harder to optimise.

Remark 4.2.10. *Depth of Network and Universal Approximation Theorem*
A feedforward network with single layer is sufficient to represent any function, but the layer may be infeasibly large and fail to learn and generalise correctly. Using deeper models can reduce number of units required to represent the desired function and can reduce generalisation error.
Shallow networks with broad family of non-polynomial activation functions have universal approximation properties. Piecewise linear networks can represent functions with number of regions that is exponential to depth. The number of linear regions carved out by deep rectifier network with d inputs, depth l, and n units per hidden layer is $O(\binom{n}{d}^{d(d-1)} n^d)$. For maxout networks with k filters per unit, this is $O(k^{(k-1)+d})$.

Remark 4.2.11. *Backpropagation*
To calculate the gradient of loss function with respect to each of individual parameters of the neural network. Model training begins with random initialisation of weights and biases.

i. Forward pass: input is sampled from training data. Nodes receive input vector and passes their value (multiplied by random initial weight) to nodes of first hidden layer. The hidden units take weighted sum of these output values as an input to the activation function, whose output is used for next hidden layer.

ii. Error computation: the final output of the network is compared to the ground truth, difference is calculated for the error value.

iii. Backwards pass: the error value computed earlier is used to compute the gradient of loss function. The gradient is then propagated back through the network, and the weights are updated according to their contribution to the error. The learning rate determines the size of weight updates.

iv. Weights update: the weights are updated in opposite direction of the gradient

Algorithm 1: Backpropagation Learning Algorithm

Require:
A set of training examples $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$
A multilayer network with L layers, weights w_{ij}^l, and activation function f
Loss function $J(y, o)$
Learning rate $0 < \alpha < 1$
Number of epochs *epochs*.

Initialize all weights:
for w_{ij}^l in the network **do**
  $w_{ij}^l \leftarrow$ small random number
end for

for $i = 1$ to *epochs* **do**
  for all training examples $(x, y) \in \mathcal{D}$ **do**

Propagate the inputs forward to compute the outputs
for all neurons i in the input layer **do**
  $a_i^0 \leftarrow x_i$
end for

for $l = 2$ to L **do**
  for all neurons i in layer l **do**
    $z_i^l \leftarrow \sum_j w_{ij}^l \, a_j^{l-1}$
    $a_i^l \leftarrow f(z_i^l)$
  end for
end for

Propagate deltas backward from output layer to input layer
for all neurone i in the output layer **do**
  Let $o_i \equiv a_i^L$ *be output of neurone* i
  $\delta_i^L \leftarrow \dfrac{\partial J(y_i, o_i)}{\partial o_i} \, f'(z_i^L)$
end for

for $l = L - 1$ to 1 **do**
  for all neurons i in layer l **do**
    $\delta_i^l \leftarrow f'(z_i^l) \sum_j (w_{ji}^{l+1} \, \delta_j^{l+1})$
  end for
end for

Update weights using the deltas
for all w_{ij}^l in the network **do**
  $w_{ij}^l \leftarrow w_{ij}^l - \alpha \, \delta_i^l \, a_j^{l-1}$
end for
end for
end for

Example 4.2.12. *Backpropagation Computation Example (Matt ?)*

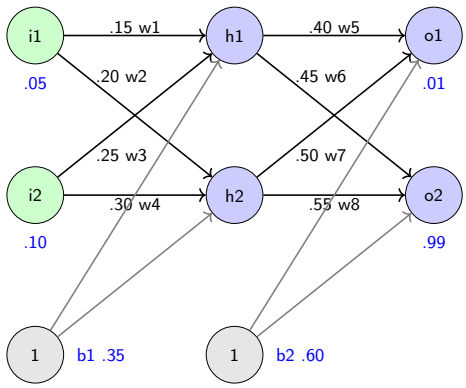


Figure 2: Backpropagation Example

For the forward pass, compute the total net input to each hidden layer neurone, squash total net input with activation function (logistic function), then repeat process with output layer neurones.
The total net input for h_1 is:

$$\begin{aligned} \text{net}_{h1} &= w_1 \cdot i_1 + w_2 \cdot i_2 + b_1 \cdot 1 \\ &= 0.15 \cdot 0.05 + 0.2 \cdot 0.1 + 0.35 \cdot 1 = 0.3775 \end{aligned}$$

Squash with logistic function to get output of h_2:

$$\text{out}_{h1} = 1/(1 + e^{-\text{net}_{h1}}) = 1/(1 + e^{-0.3775}) = 0.593269992$$

For h_2, repeating the same process, the output is then $\text{out}_{h2} = 0.596884378$.
Repeating the process for outer layer neurones,

$$\begin{aligned}\text{net}_{o1} &= w_5 \cdot \text{out}_{h1} + w_6 \cdot \text{out}_{h2} + b_2 \cdot 1 \\ &= 0.4 \cdot 0.593269992 + 0.45 \cdot 0.596884378 + 0.6 \cdot 1 = 1.105905967 \\ \text{out}_{o1} &= 1/(1 + e^{-\text{net}_{o1}}) = 1/(1 + e^{-1.105905967}) = 0.75136507\end{aligned}$$

For o_2, repeating the same process, the output is then $\text{out}_{o2} = 0.772928465$.
The total error for each output neurone may be computed using squared error function $E_{\text{total}} = \sum \frac{1}{2}(Y - \hat{Y})^2$.
For o_1, the target output is 0.01, neurone output is 0.75136507, hence error is

$$E_{o1} = \frac{1}{2}(0.01 - 0.75136507)^2 = 0.274811083$$

Repeating the process for o_2, the total error for neural network is then

$$E_{\text{total}} = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$

For the backpropagation step, update each of the weights in network to reduce the error for each output neurone.

$$\begin{aligned}E_{\text{total}} &= \frac{1}{2}(Y_{o1} - \hat{Y}_{o1})^2 + \frac{1}{2}(Y_{o2} - \hat{Y}_{o2})^2 \\ \frac{\partial E_{\text{total}}}{\partial \text{out}_{o1}} &= 2 \cdot \frac{1}{2}(Y_{o1} - \hat{Y}_{o1}) \cdot -1 + 0 = -(0.01 - 0.75136507) = 0.74136507\end{aligned}$$

Also, note that

$$\begin{aligned}\text{out}_{o1} &= 1/(1 + e^{-\text{net}_{o1}}) \\ \frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} &= \text{out}_{o1}(1 - \text{out}_{o1}) = 0.75136507(1 - 0.75136507) = 0.186815602 \\ \text{net}_{o1} &= w_5 \cdot \text{out}_{h1} + w_6 \cdot \text{out}_{h2} + b_2 \cdot 1 \\ \frac{\text{net}_{o1}}{w_5} &= 1 \cdot \text{out}_{h1} \cdot w_5^0 + 0 + 0 = \text{out}_{h1} = 0.593269992\end{aligned}$$

Putting it all together,

$$\begin{aligned}\frac{\partial E_{\text{total}}}{\partial w_5} &= \frac{\partial E_{\text{total}}}{\partial \text{out}_{o1}} \cdot \frac{\partial \text{out}_{o1}}{\partial \text{net}_{o1}} \cdot \frac{\partial \text{out}_{o1}}{\partial w_5} \\ &= -(Y_{o1} - \hat{Y}_{o1}) \cdot \text{out}_{o1}(1 - \text{out}_{o1}) \cdot textout_{o1} \\ &= 0.74136507 \cdot 0.186815602 \cdot 0.593269992 = 0.0821267041\end{aligned}$$

To decrease the error, subtract the value from current weight multiplied by learning rate (i.e., $\alpha = 0.5$).

$$w_5^+ = w_5 - \alpha \frac{\partial E_{\text{total}}}{\partial w_5} = 0.4 - 0.5 \cdot 0.0821267041$$

Repeating for the rest of the weights, we get

$$w_6^+ = 0.408666186, \quad w_7^+ = 0.511301270, \quad w_8^+ = 0.561370121$$

For the hidden layers, note that

$$\begin{aligned}\frac{\partial E_{\text{total}}}{\partial w_1} &= \frac{\partial E_{\text{total}}}{\partial \text{out}_{h1}} \cdot \frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} \cdot \frac{\partial \text{net}_{h1}}{\partial w_1} \\ &= \left(\frac{\partial E_{o1}}{\partial \text{out}_{h1}} + \frac{\partial E_{o2}}{\partial \text{out}_{h1}}\right) \cdot \frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} \cdot \frac{\partial \text{net}_{h1}}{\partial w_1}\end{aligned}$$

Note that out_{h1} affects both out_{o1} and out_{o2}.

$$\frac{\partial E_{o1}}{\partial \text{out}_{h1}} = \frac{\partial E_{o1}}{\partial \text{net}_{h1}} \cdot \frac{\partial \text{net}_{h1}}{\partial \text{out}_{h1}}$$
$$\frac{\partial E_{o1}}{\partial \text{net}_{h1}} = \frac{\partial E_{o1}}{\partial \text{out}_{h1}} \cdot \frac{\partial \text{out}_{o1}}{\partial \text{net}_{h1}} = 0.74136507 \cdot 0.186815602 = 0.138498562$$
$$\frac{\partial \text{net}_{h1}}{\partial \text{out}_{h1}} = w_5 = 0.4$$
$$\frac{\partial E_{o1}}{\partial \text{out}_{h1}} = 0.138498562 \cdot 0.4 = 0.055399425$$

Following same process, $\frac{\partial E_{o2}}{\partial \text{out}_{h1}} = -0.019049119$. Hence

$$\frac{\partial E_{\text{total}}}{\partial \text{out}_{h1}} = \frac{\partial E_{o1}}{\partial \text{out}_{h1}} + \frac{\partial E_{o2}}{\partial \text{out}_{h1}} = 0.05539425 - 0.019049119 = 0.036350306$$

Next, figure out $\frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}}$ and $\frac{\partial \text{net}_{h1}}{\partial w_1}$.

$$\frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} = \text{out}_{h1} \cdot (1 - \text{out}_{h1}) = 0.59326999(1 - 0.59326999) = 0.2410.241300709$$
$$\frac{\partial \text{net}_{h1}}{\partial w_1} = i_1 = 0.05$$

Putting it together,

$$\frac{\partial E_{\text{total}}}{\partial w_1} = \frac{\partial E_{\text{total}}}{\partial \text{out}_{h1}} \cdot \frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} \cdot \frac{\partial \text{net}_{h1}}{\partial w_1} = 0.036350306 \cdot 0.241300709 \cdot 0.05 = 0.000438658$$

This is also written as

$$\frac{\partial E_{\text{total}}}{\partial w_1} = \left(\sum_o \frac{\partial E_{\text{total}}}{\partial \text{out}_o} \cdot \frac{\partial \text{out}_o}{\partial \text{net}_o} \cdot \frac{\partial \text{net}_o}{\partial \text{out}_{h_1}} \right) \cdot \frac{\partial \text{out}_{h1}}{\partial \text{net}_{h1}} \cdot \frac{\partial \text{net}_{h1}}{\partial w_1}$$

Next, update w_1 and rest of the weights

$$w_1^+ = w_1 - \alpha \frac{\partial E_{\text{total}}}{\partial w_1} = 0.15 - 0.5 \cdot 0.000438658 = 0.149780716$$
$$w_2^+ = 0.19956143, \quad w_3^+ = 0.24975114, \quad w_4^+ = 0.29950229$$

Hence all the weights has been updated. After the first round of backpropagation, total error has decreased to 0.291027924. Repeating the process $10,000$ times, the error reduces to 0.0000351085.

4.3 Regularisation for Deep Learning

Definition 4.3.1. *Regularisation* refers to adding a parameter norm penalty $\Omega(\boldsymbol{\theta})$ to the objective function J. The regularised objective function is then

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha\Omega(\boldsymbol{\theta})$$

where $\alpha in [0, \infty)$ is a hyper-parameter that weights the contribution of norm penalty term.
For neural networks, the parameter norm penalty Ω is chosen such that it penalises only the weights of the affine transformation at each layer and leaves the biases un-regularised.

Definition 4.3.2. *L^2 (Ridge) Regularisation*
The regularisation term added to objective function is

$$\Omega(\boldsymbol{\theta}) = \frac{1}{2}\|\boldsymbol{w}\|_2^2$$

Remark 4.3.3. *Behaviour of Weight Decay (L^2) Regularisation*
Assuming no bias parameter, a model have the following objective function and parameter gradient:

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \frac{\alpha}{2}\boldsymbol{w}^T\boldsymbol{w} + J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$
$$\nabla_{\boldsymbol{w}}\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha\boldsymbol{w} + \nabla_{\boldsymbol{w}}J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

On a single gradient step, the update is as follows:

$$\boldsymbol{w} \leftarrow (1 - \epsilon\alpha)\boldsymbol{w} - \epsilon\nabla_{\boldsymbol{w}}J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

The addition of weight decay has modified learning rule to multiplicatively shrink weight vector by constant factor on each step before gradient update.
Using quadratic approximation to objective function at minimal unregularised training cost, approximation is

$$\hat{J}(\bar{\theta}) = J(\boldsymbol{w}^*) + \frac{1}{2}(\boldsymbol{w} - \boldsymbol{w}^*)\boldsymbol{H}(\boldsymbol{w} - \boldsymbol{w}^*), \qquad \boldsymbol{w}^* = \arg\min_{\boldsymbol{w}} J(\boldsymbol{w})$$

where $\boldsymbol{H}$ is Hessian matrix of J with respect to $\boldsymbol{w}$ evaluated at $\boldsymbol{w}^*$. Minimum of $\hat{J}$ occurs where gradient is

$$\nabla_{\boldsymbol{w}}\hat{J}(\boldsymbol{w}) = \boldsymbol{H}(\boldsymbol{w} - \boldsymbol{w}^*) = \boldsymbol{0}$$

Adding weight decay gradient, solve for minimum of regularised version of $\hat{J}$. Let $\tilde{\boldsymbol{w}}$ be minimum, then

$$\alpha\tilde{\boldsymbol{w}} + \boldsymbol{H}(\tilde{\boldsymbol{w}} - \boldsymbol{w}^*) = 0$$
$$(\boldsymbol{H} + \alpha\boldsymbol{I})\tilde{\boldsymbol{w}} = \boldsymbol{H}\boldsymbol{w}^*$$
$$\tilde{\boldsymbol{w}} = (\boldsymbol{H} + \alpha\boldsymbol{I})^{-1}\boldsymbol{H}\boldsymbol{w}^*$$

As $\alpha \to 0$, regularised solution $\tilde{\boldsymbol{w}} \to \boldsymbol{w}^*$. Note $\boldsymbol{H}$ is real and symmetric, hence decompose into diagonal matrix $\boldsymbol{A}$ and orthonormal basis of eigenvectors $\boldsymbol{Q}$ such that $\boldsymbol{H} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T$, to get

$$\tilde{\boldsymbol{w}} = (\boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T + \alpha\boldsymbol{I})^{-1}\boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T\boldsymbol{w}^* = \boldsymbol{Q}(\boldsymbol{\Lambda} + \alpha\boldsymbol{I})^{-1}\boldsymbol{\Lambda}\boldsymbol{Q}^T\boldsymbol{w}^*$$

The effect of weight decay rescale $\boldsymbol{w}^*$ along axes defined by eigenvectors of $\boldsymbol{H}$. The component of $\boldsymbol{w}^*$ aligned with i-th eigenvector of $\boldsymbol{H}$ is rescaled by factor $\frac{\lambda_i}{\lambda_i + \alpha}$. For components where $\lambda_i \gg \alpha$, the effects of regularisation is relatively small. For components where $\lambda_i \ll \alpha$, components will be shrunk to nearly zero magnitude.

Definition 4.3.4. *L^1 Regularisation*
The L^1 regularisation is the sum of absolute values of individual parameters.

$$\Omega(\boldsymbol{\theta}) = \|\boldsymbol{w}\|_1 = \sum_i |w_i|$$

Definition 4.3.5. *Behaviour of L^1 Regularisation*
Assuming no bias parameter, a model has the following objective function and parameter gradient:

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha\|\boldsymbol{w}\|_1 + J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$
$$\nabla_{\boldsymbol{w}}\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha\ \text{sign}(\boldsymbol{w}) + \nabla_{\boldsymbol{w}}J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

Note that the regularisation contribution to gradient is a constant factor with sign equal to $\text{sign}(w_i)$. Minimum of $\tilde{J}$ occurs at where

$$\nabla_{\boldsymbol{w}} \tilde{J}(\boldsymbol{w}) = \boldsymbol{H}(\boldsymbol{w} - \boldsymbol{w}^*)$$

Assuming the Hessian is diagonal, $\boldsymbol{H} = \text{diag}([H_{1,1}, \dots, H_{n,n}])$, where each $H_{i,i} > 0$. The quadratic approximation of regularised objective function is then

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{w}^*; \boldsymbol{X}, \boldsymbol{y}) + \sum_i \left[\frac{1}{2} H_{i,i} (\boldsymbol{w}_i - \boldsymbol{w}_i^*)^2 + \alpha \left| w_i \right| \right]$$

$$w_i = \text{sign}(w_i^*) \max \left\{ |w_i^*| - \frac{\alpha}{H_{i,i}}, 0 \right\}$$

For the case where $w_i^* > 0$ for all i, then

i. if $w_i^* \leq \frac{\alpha}{H_{i,i}}$, the optimal value is $w_i = 0$ as contribution of $J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$ to regularised objective $\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$ is overwhelmed in direction i by L^1 regularisation which pushes w_i to zero.

ii. if $w_i^* > \frac{\alpha}{H_{i,i}}$, regularisation shifts optimal value of w_i in the direction by $\frac{\alpha}{H_{i,i}}$.

For the case where $w_i^* < 0$, this happens similarly, but with L^1 penalty decreasing w_i by $\frac{\alpha}{H_{i,i}}$, with min value 0. Note that L^1 produces a more sparse solution, which is a feature selection mechanism.

Remark 4.3.6. *Norm Penalties as Constrained Optimisation*
Let cost function regularised by parameter norm penalty be

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha \Omega(\boldsymbol{\theta})$$

Construct a generalised generalised Lagrange function,

$$\mathcal{L}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha(\Omega(\boldsymbol{\theta}) - k)$$

The solution to the constrained problem is then

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \max_{\alpha, \alpha \geq 0} \mathcal{L}(\boldsymbol{\theta}, \alpha)$$

Note that optimal value α^* will shrink $\Omega(\boldsymbol{\theta})$, but not such that it is less than k.
Fixing α^*, the problem is then a function of $\boldsymbol{\theta}$,

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}, \alpha^*) = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha^* \Omega(\boldsymbol{\theta})$$

This is the regularised training problem of minimising $\tilde{J}$. The parameter norm penalty is imposing a constraint on the weights. If explicit constraints are to be used rather than penalties, use stochastic gradient descent on $J(\boldsymbol{\theta})$, then take the projected $\boldsymbol{\theta}$ to the nearest point that satisfies $\Omega(\boldsymbol{\theta}) < k$. This method is used if k is predefined by user, and time is not to be spent on searching for α value that corresponds to this k.
Explicit constraints and re-projection work better in circumstances where non-convex optimisation is involved, and this may avoid getting stuck in local minima. Explicit constraints also impose stability, and together with high learning rate allows rapid exploration of parameter space.

Remark 4.3.7. *Dataset Augmentation*
Effective technique for object classification. Operations such as translation of images by few pixels in each direction can greatly improved generalisation, even if the model has been designed to be partially translation invariant using convolution and pooling techniques. Rotation and scaling of images are also quite effective.
For speech recognition tasks, noice injection will improve performance. To improve robustness, train models with random noise applied to inputs. Noise injection also works when applied to the hidden units (data augmentation at multiple levels of abstraction). Enabling dropout constructs new inputs by multiplying by noise.

Remark 4.3.8. *Noise Robustness*
For some models, addition of noise with infinitesimal variance at input of model is equivalent to imposing penalty on norm of the weights. Generally, noise injection can be much more powerful than simply shrinking the parameters, especially when noise is added to hidden units.
Noise may also be added to weights, which is primarily used in recurrent neural networks. This is a stochastic implementation of Bayesian inference over the weights (model weights are uncertain and representable via a probability distribution that reflects this uncertainty).

Adding noise to weights may also result in more stability in function to be learned. Let the cost function be

$$J = \mathbb{E}_{p(x,y)}[(\hat{y}(\boldsymbol{x}) - y)^2]$$

where $\hat{y}(\boldsymbol{x})$ is function to be trained that maps set of features $\boldsymbol{x}$ to a scalar using least-squares cost function J. Assume each input presentation include a random perturbation $\epsilon_{\boldsymbol{W}} \sim \mathcal{N}(\boldsymbol{\epsilon}; \boldsymbol{0}, \eta \boldsymbol{I})$ in the network weight of a standard l-layer MLP. Let the perturbed model be $\hat{y}_{\boldsymbol{\epsilon_W}}(\boldsymbol{x})$. With injection o noise, objective function becomes

$$\tilde{J}_{\boldsymbol{W}} = \mathbb{E}_{p(\boldsymbol{x},y,\boldsymbol{\epsilon_W})}\left[(\hat{y}_{\boldsymbol{\epsilon_W}}(\boldsymbol{x}) - y)^2\right] = \mathbb{E}_{p(\boldsymbol{x},y,\boldsymbol{\epsilon_W})}\left[\hat{y}^2_{\boldsymbol{\epsilon_W}}(\boldsymbol{x}) - 2y\hat{y}_{\boldsymbol{\epsilon_W}} + y^2\right]$$

For small η, the minimisation of J with added noise (with covariance $\eta \boldsymbol{I}$) is equivalent to minimisation of J with additional regularisation term $\eta \mathbb{E}_{p(\boldsymbol{x}),y}[\|\nabla_{\boldsymbol{W}} \hat{y}(\boldsymbol{x})\|^2]$. This encourages parameters to go to regions of parameter space where small perturbations of weights have relatively small influence on output.

Remark 4.3.9. *Label Smoothing*
Datasets may have some mistake in y labels. Regularise a model based on softmax with k output values by replacing hard 0 and 1 classification targets with targets of $\frac{\epsilon}{k-1}$ and $1-\epsilon$ respectively, assuming that the training set label y is correct with probability $1 - \epsilon$. Cross-entropy can then be used on these soft targets.

Remark 4.3.10. *Semi-Supervised Learning*
Unlabelled examples $P(\boldsymbol{x})$ and $P(\boldsymbol{x}, \boldsymbol{y})$ are used to estimate $P(\boldsymbol{y} \mid \boldsymbol{x})$ or predict $\boldsymbol{y}$ from $\boldsymbol{x}$.
Construct a generative model of either $P(\boldsymbol{x})$ or $P(\boldsymbol{x}, \boldsymbol{y})$ which shares parameters with a discriminative model of $P(\boldsymbol{y} \mid \boldsymbol{x})$. The supervised criterion $\log P(\boldsymbol{y} \mid \boldsymbol{x})$ may then be traded with the unsupervised or generative one ($\log P(\boldsymbol{x})$ or $\log P(\boldsymbol{x}, \boldsymbol{y})$). The generative criterion then expresses a particular form of prior belief about the solution to the supervised learning problem (structure of $P(\boldsymbol{x})$ is connected to structure of $P(\boldsymbol{y} \mid \boldsymbol{x})$ such that is is captured by the shared parametrisation).

Remark 4.3.11. *Multi-Task Learning*
Improve generalisations by pooling examples (soft constraints imposed on parameters) from several tasks.
Different supervised tasks (predicting $\boldsymbol{y}^{(i)}$ given $\boldsymbol{x}$) share the same input $\boldsymbol{x}$, as well as some intermediate-level representation $\boldsymbol{h}^{(\text{shared})}$ capturing a common pool of factors. Model can be divided in two parts:

i. Task-specific parameters
ii. Generic parameters, shared across all the tasks

Improved generalisation and associated error bounds can be achieved due to shared parameters, but this only happens if some assumptions about statistical relationship between different tasks are valid.

Remark 4.3.12. *Early Stopping*
To obtain a model with better validation set error by returning to the parameter setting at the point in time with lowest validation set error. Model validation is done in parallel to model training.
Early stopping has the effect of restricting the optimisation procedure to a relatively small volume of parameter space in neighbourhood of initial parameter value $\boldsymbol{\theta}_o$. Assume τ optimisation steps are taken with learning rate ϵ. The product $\epsilon\tau$ is a measure of effective capacity. Assuming gradient is bounded, restricting $\epsilon\tau$ limits the volume of parameter space reachable from $\boldsymbol{\theta}_o$, hence $\epsilon\tau$ behaves as the reciprocal of weight decay coefficients.

Algorithm 2: Early Stopping Algorithm

Require:
Number of steps n between evaluations
Patience p, the number of times to observe worsening validation error before stopping
Initial parameters $\boldsymbol{\theta}_o$

$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_o$, $i \leftarrow 0$, $j \leftarrow 0$, $v \leftarrow \infty$, $\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}$, $i^* \leftarrow i$
while $j < p$ **do**
  Update $\boldsymbol{\theta}$ by running training algorithm for n steps
  $i \leftarrow i + n$, $v' \leftarrow$ ValidationSetError($\boldsymbol{\theta}$)
  if v' < v **then**
    $j \leftarrow 0$, $\boldsymbol{\theta}^* \leftarrow \boldsymbol{\theta}$, $i^* \leftarrow i$, $v \leftarrow v'$
  end if
  $j \leftarrow j + 1$
end while
Best parameters are $\boldsymbol{\theta}^*$, best number of training steps is i^*

Remark 4.3.13. *Parameter Tying and Parameter Sharing*
Used to express prior knowledge on suitable values of model parameters (precise values are not known, but there should be some dependencies between parameters). To regularise parameters to be close, may use:

i. Parameter norm penalty: assume model A with parameters $\boldsymbol{w}^{(A)}$ and model B with parameters $\boldsymbol{w}^{(B)}$. Two models map input to two different but related outputs $\hat{y}^{(A)} = f(\boldsymbol{w}^{(A)}, \boldsymbol{x})$ and $\hat{y}^{(B)} = g(\boldsymbol{w}^{(B)}, \boldsymbol{x})$. The model parameters $w_i^{(A)}$ should be close to $w_i^{(B)}$ $\forall i$. Using regularisation, a parameter norm penalty may be imposed, such as using L^2 penalty to get $\Omega(\boldsymbol{w}^{(A)}, \boldsymbol{w}^{(B)}) = \left\|\boldsymbol{w}^{(A)} - \boldsymbol{w}^{(B)}\right\|_2^2$.
ii. Parameter sharing: using constraints. Only a subset of the parameters (input set) need to be stored in memory. May lead to significant reduction in memory footprint of the model (i.e., in CCNs)

Remark 4.3.14. *Sparse Representations*
Representation sparsity is where elements of the representation are zero, accomplished by same sort of mechanisms used in parameter regularisation.
Given a linear regression $\boldsymbol{y} = \boldsymbol{B}\boldsymbol{h}$, where $\boldsymbol{y} \in \mathbb{R}^m, \boldsymbol{B} \in \mathbb{R}^{m \times n}, \boldsymbol{h} \in \mathbb{R}^n$, and $\boldsymbol{h}$ is the representation of data $\boldsymbol{x}$, add to loss function J a norm penalty on the representation ($\Omega(\boldsymbol{h})$)):

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha\Omega(\boldsymbol{h})$$

where $\alpha \in [0, \infty)$ weights relative contribution of norm penalty, with larger values of α for more regularisation. L^1 penalty on elements of representation induces sparsity, $\Omega(\boldsymbol{h}) = \|\boldsymbol{h}\|_1 = \sum_i |h_i|$. For representations with elements constrained to lie on unit interval, may use Student-t prior or KL divergence penalties.
Other approaches such as hard constraint on activation values may also work. *Orthogonal matching pursuit* encodes an input $\boldsymbol{x}$ with representation $\boldsymbol{h}$ that solves the constrained optimisation problem

$$\arg \min_{\boldsymbol{h}, \|\boldsymbol{h}\|_0 < k} \|\boldsymbol{x} - \boldsymbol{W}\boldsymbol{h}\|^2$$

where $\|\boldsymbol{h}\|_0$ is the number of non-zero entries of $\boldsymbol{h}$. If $\boldsymbol{W}$ is constrained to be orthogonal, this can be solved efficiently with OMP-k method, where k is number of non-zero features allowed.

Remark 4.3.15. *Bootstrap Aggregating (Bagging)*
Construct k different datasets, each with same number of examples as original dataset, but constructed by sampling with replacement from original dataset. Each dataset may miss some examples but also contain duplicate examples. Each model i is then trained on the dataset i. Differences in random initialisation, random selection of mini-batches, differences in hyperparameters, or different outcomes of non-deterministic implementation of neural networks are enough to allow each model to make partially independent errors.

Remark 4.3.16. *Boosting*
Construct k different datasets from original dataset. Using ith dataset, train ith weak learner. Test the ith weak learner with validation dataset, and each datapoint with wrong prediction is sent to $(i+1)$th dataset. The $(i+1)$th dataset is then used to train the $(i+1)$th weak learner. Continue until reaching the total number of subsets for the total prediction.

Remark 4.3.17. *Dropout*
Dropout trains the ensemble consisting of all sub-networks that can be formed by removing non-output units from the base network by multiplying its output value by zero. To train a neural network with dropout, use minibatch-based learning algorithms (i.e., Stochastic Gradient Descent). Each time an example is loaded into minibatch, randomly (and independently) sample a different binary mask $\boldsymbol{\mu}$ to apply to all of input and hidden units in the network, where the probability of sampling a mask value of one is the dropout rate.
Let $J(\boldsymbol{\theta}, \boldsymbol{\mu})$ be cost of model defined by parameters $\boldsymbol{\theta}$ and mask $\boldsymbol{\mu}$. Dropout training is minimising $\mathbb{E}_{\boldsymbol{\mu}} J(\boldsymbol{\theta}, \boldsymbol{\mu})$. Each model inherits different subset of parameters from parent neural network, and a tiny fraction of possible sub-networks are trained for a single step. Parameter sharing allows sub-networks to arrive at good parameter settings. Each sub-network given by $\boldsymbol{\mu}$ defines a probability distribution $p(y \mid \boldsymbol{x}, \boldsymbol{\mu})$. The arithmetic mean over all masks is then given by

$$\sum_{\boldsymbol{\mu}} p(\boldsymbol{\mu}) p(y \mid \boldsymbol{x}, \boldsymbol{\mu})$$

where $p(\boldsymbol{\mu})$ is the distribution used to sample $\boldsymbol{\mu}$ during training.
Using dropout as a regularisation technique reduces effective capacity of a model. To offset this, increase the size of the model. Typically optimal validation error is much lower when using dropout, but this comes at cost of larger model and more iterations of training.
For very large datasets, costs of using dropout and larger models may outweigh benefits of regularisation.

For sparse datasets (extremely few labelled training examples available), Bayesian neural networks outperform dropout. When additional unlabelled data is available, unsupervised learning may gain advantage over dropout.

Remark 4.3.18. *Adversarial Training*
Adversarial example may be constructed by optimisation to search for an input $\boldsymbol{x}'$ near $\boldsymbol{x}$ such that the model output is very different at $\boldsymbol{x}'$.
The primary cause is excessive linearity in some neural networks. Changing input by ϵ causes linear function with weights $\boldsymbol{w}$ to change by as much as $\epsilon \|\boldsymbol{w}\|_1$, which can be very large if $\boldsymbol{w}$ is high-dimensional.
Adversarial examples assists in semi-supervised learning. At point $\boldsymbol{x}$ that is not associated with a label, model assigns label $\hat{y}$ (may not be true label). Seek adversarial example $\boldsymbol{x}'$ that causes classifier to output label $y' \neq \hat{y}$. Classifier may then be trained to assign same label to $\boldsymbol{x}$ and $\boldsymbol{x}'$, learning function robust to small changes.

Remark 4.3.19. *Tangent Distance Algorithm*
Non-parametric nearest-neighbour algorithm, where examples on the same manifold share the same category. The nearest-neighbour distance between points $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ is the distance between manifolds M_1 and M_2. Computationally cheap alternative is to approximate M_i by its tangent plane at $\boldsymbol{x}_i$ and measure the distance between two tangents, or between a tangent plane and a point.

Remark 4.3.20. *Tangent Propagation Algorithm*
Trains neural net classifier with extra penalty to make each output $f(\boldsymbol{x})$ of neural net locally invariant to factors of variation, which correspond to movement along manifold which examples from same class concentrate. Local invariance is achieved by requiring $\nabla_{\mathbf{x}} f(\boldsymbol{x})$ to be orthogonal to known manifold tangent vectors $\boldsymbol{v}^{(i)}$ at $\boldsymbol{x}$, or equivalently the directional derivative of f at $\boldsymbol{x}$ in directions $\boldsymbol{v}^{(i)}$ be small by adding regularisation penalty:

$$\Omega(f) = \sum_i \left(\left(\nabla_{\boldsymbol{x}} f(\boldsymbol{x}) \right)^T \boldsymbol{v}^{(i)} \right)^2$$

Tangent vectors are derived a priori from effect of transformations (i.e., translation, rotation, scaling).
Note this regularises model to resist infinitesimal perturbation, and the infinitesimal approach poses difficulties for models based on rectified linear units.
Data augmentation is the non-infinitesimal version of Tangent Propagation Algorithm, where network is explicitly trained to correctly classify distinct inputs created from more than infinitesimal transformation.

Remark 4.3.21. *Double Back-propagation Algorithm*
Algorithm regularises Jacobian to be small. Require model to be invariant to all directions of change in input so long as the change is small. Adversarial training is the non-infinitesimal version of Double Back-propagation Algorithm, where inputs were found near original inputs and model is trained to produce same input.

Remark 4.3.22. *Manifold Tangent Classifier*
Eliminates need to know tangent vectors a priori. Auto-encoders may estimate manifold tangent vectors, and the classifier makes use of this technique to avoid requiring user-specified tangent vectors. The steps are:

i. Use auto-encoder to learn manifold structure by unsupervised learning
ii. Use these tangents to regularise a neural net classifier as in Tangent Propagation Algorithm.

4.3.1 Optimisation for Deep Learning

Remark 4.3.23. *Optimisation*
A cost function $J(\boldsymbol{\theta})$ is reduced to improve some performance measure P.
Optimisation algorithms include some specialisation on specific structure of machine learning objective functions.
Cost function is average over training set,

$$J(\boldsymbol{\theta}) = \mathbb{E}_{(\boldsymbol{x},y)\sim\hat{p}_{\text{data}}} L(f(\boldsymbol{x};\boldsymbol{\theta}), y)$$

where L is per-example loss function, $f(\boldsymbol{x};\boldsymbol{\theta})$ is predicted output given input $\boldsymbol{x}$, $\hat{p}_{\text{data}}$ is empirical distribution.
For supervised learning, y is target output.
The goal is to usually to minimise objective function where expectation is taken on data generating distribution p_{data} rather than over the finite training set:

$$J^*(\boldsymbol{\theta}) = \mathbb{E}_{(\boldsymbol{x},y)\sim p_{\text{data}}} L(f(\boldsymbol{x};\boldsymbol{\theta}), y)$$

Note that machine learning algorithm usually minimises a surrogate loss function, and halts when convergence criterion based on early stopping is satisfied.

Definition 4.3.24. *Empirical Risk Minimisation*
To minimise expected loss on training set, where $\hat{p}(\boldsymbol{x}, y)$ is the empirical distribution. The empirical risk is

$$\mathbb{E}_{\boldsymbol{x},y\sim\hat{p}_{\text{data}}(\boldsymbol{x},y)}[L(f(\boldsymbol{x},\boldsymbol{\theta}), y)] = \frac{1}{m}\sum_{i=1}^{m} L(f(\boldsymbol{x}^{(i)};\boldsymbol{\theta}), y^{(i)})$$

where m is number of training samples. Note that empirical risk minimisation is prone to overfitting, as models with high capacity may memorise the training set.

Remark 4.3.25. *Batch and Minibatch Algorithms*
Objective algorithms for machine learning computes each update to the parameters based on expected value of cost function using only a subset of terms of full cost function.
For maximum likelihood estimation problems, note that

$$\boldsymbol{\theta}_{\text{ML}} = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{m} \log p_{\text{model}}(\boldsymbol{x}^{(i)}, y^{(i)}; \boldsymbol{\theta})$$
$$J(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{x},y\sim\hat{p}_{\text{data}}} \log p_{\text{model}}(\boldsymbol{x}, y; \boldsymbol{\theta})$$

The gradient is then

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{x},y\sim\hat{p}_{\text{data}}} \nabla_{\boldsymbol{\theta}} \log\log p_{\text{model}}(\boldsymbol{x}, y; \boldsymbol{\theta})$$

If the optimisation algorithm uses the entire training set, then it is a batch gradient method.
If a single sample is used at a time, then this is a stochastic/online gradient method.
For algorithms that uses more than one but less than all samples, then this is a minibatch gradient method.

Remark 4.3.26. *Challenges: Ill-Conditioning of Hessian Matrix H*
May cause stochastic gradient descent to be stuck where very small step increases the cost function.
Note that a gradient descent step of $-\boldsymbol{\epsilon g}$ will add

$$\frac{1}{2}\epsilon^2 \boldsymbol{g}^T \boldsymbol{H}\boldsymbol{g} - \epsilon \boldsymbol{g}^T \boldsymbol{g}$$

to the cost. This will be a problem when $\frac{1}{2}\epsilon^2 \boldsymbol{g}^T \boldsymbol{H}\boldsymbol{g} > \epsilon \boldsymbol{g}^T \boldsymbol{g}$.
Monitor squared gradient norm $\boldsymbol{g}^T\boldsymbol{g}$ and $\boldsymbol{g}^T\boldsymbol{H}\boldsymbol{g}$ term. When ill-conditioning happens, gradient norm does not shrink significantly but $\boldsymbol{g}^T\boldsymbol{H}\boldsymbol{g}$ term grows more than order of magnitude, and learning rate shrinks.

Remark 4.3.27. *Challenges: Local Minimums*
Neural networks and models with multiple parametrised latent variables have multiple local minima due to *model identifiability* problem. Model is identifiable if a sufficiently large training set can rule out all but one setting of model parameters. Models with latent variables are not identifiable as equivalent models are obtained by exchanging latent variables with each other (*weight space symmetry*). If there are m layers with n units each, then there are $n!^m$ ways to arrange the hidden units, making the model unidentifiable.
There may be an extremely large or uncountably infinite amount of local minima in a cost function. If the local minima have a high cost compared to global minimum, then this is problematic.

Remark 4.3.28. *Challenges: Plateaus, Saddle Points, Flat Regions*
For a function $f : \mathbb{R}^n \to \mathbb{R}$, expected ratio of saddle points to local minima grows exponentially with n. Hessian

matrix at a saddle point has a mixture of positive and negative eigenvalues.
Degenerate regions with flat gradient may correspond to high value of objective function.

Remark 4.3.29. *Challenges: Cliffs and Exploding Gradients*
On extremely steep cliff structure, gradient update step may move parameters extremely far, losing most of the optimisation work done previously. The consequences may be mitigated using gradient clipping heuristics, which reduces the step size to be small enough.

Remark 4.3.30. *Challenges: Long-Term Dependencies*
Occurs when computational graph becomes very deep. Let a computational graph contain a path that repeatedly multiply a matrix $\boldsymbol{W}$. After t steps, this is $\boldsymbol{W}^t$. Let the eigen-decomposition of $\boldsymbol{W}$ be $\boldsymbol{W} = \boldsymbol{V}\text{diag}(\boldsymbol{\lambda})\boldsymbol{V}^{-1}$. Then we can see that $\boldsymbol{W}^t = (\boldsymbol{V}\text{diag}(\boldsymbol{\lambda})\boldsymbol{V}^{-1})^t = \boldsymbol{V}\text{diag}(\lambda)^t\boldsymbol{V}^{-1}$.
There will be a vanishing and exploding gradient problem if gradients on the graph are scaled according to $\text{diag}(\boldsymbol{\lambda})^t$. Note that recurrent networks uses same $\boldsymbol{W}$ at each time step, hence may face the problem.

Remark 4.3.31. *Challenges: Inexact Gradients*
Objective function for minimisation is intractable due to noisy/biased estimate of these quantities; in this case its gradient is intractable as well. Choose surrogate loss function that is easier to approximate than true loss.

Remark 4.3.32. *Challenges: Poor Correspondence between Local and Global Structure*
As gradient descent algorithms make small local moves, optimisation on local downhill moves may fail if local surface does not point toward the global solution. In higher dimensional space, learning algorithms may incur high costs or long training time to circumnavigate the manifold. Hence to find good initial points for problems.

Remark 4.3.33. *Stochastic Gradient Descent (SGD)*
Crucial parameter is the learning rate, which may be changed over time; at iteration k this is ϵ_k.
The SGD gradient estimation introduces a source of noise that does not vanish even at minimum. A sufficient condition to guarantee converge of SGD is

$$\sum_{k=1}^{\infty} \epsilon_k = \infty, \quad \sum_{k=1}^{\infty} \epsilon_k^2 < \infty$$

In practice, it is common to decay learning rate linearly until iteration τ:

$$\epsilon_k = (1-\alpha)\epsilon_0 + \alpha\epsilon_\tau, \quad \alpha = \frac{k}{\tau}$$

After iteration τ, leave ϵ constant.
Compute time per update does not growth with number of training examples.

Algorithm 3: Stochastic Gradient Descent (SGD) Update at Training Iteration k

Require:
Learning rate ϵ_k, Initial parameter $\boldsymbol{\theta}$

while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute gradient estimate $\hat{\boldsymbol{g}} \leftarrow +\frac{1}{m}\nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon\hat{\boldsymbol{g}}$
end while

Remark 4.3.34. *Momentum Algorithm*
Algorithm introduces variable $\boldsymbol{v}$ that is set to an exponentially decaying average of negative gradient.
Hyperparameter $\alpha \in [0, 1)$ determines how quickly the contributions of previous gradients exponentially decay.
The update rule is given by

$$\boldsymbol{v} \leftarrow \alpha\boldsymbol{v} - \epsilon\nabla_{\boldsymbol{\theta}} \left(\frac{1}{m} \sum_{i=1}^{m} L(\boldsymbol{f}(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)}) \right)$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$$

The larger α is relative to ϵ, the more previous gradients affect current direction.
If gradient is $\boldsymbol{g}$, it will accelerate in direction of $-\boldsymbol{g}$ until reaching terminal velocity where size of each step is

$$\frac{\epsilon \|\boldsymbol{g}\|}{1-\alpha}$$

Algorithm 4: Stochastic Gradient Descent (SGD) with Momentum

Require:
Learning rate ϵ_k, Momentum parameter α

while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute gradient estimate $\boldsymbol{g} \leftarrow +\frac{1}{m}\nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
end while

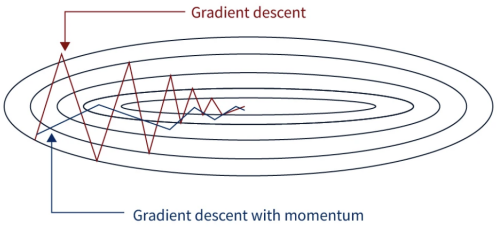


Figure 3: Stochastic Gradient Descent (SGD) Algorithm with and without Momentum

Remark 4.3.35. *Nesterov Momentum*
Inspired by Nesterov's accelerated gradient method, the update rules are:

$$\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \nabla_{\boldsymbol{\theta}} \left(\frac{1}{m} \sum_{i=1}^{m} L(\boldsymbol{f}(\boldsymbol{x}^{(i)}; \boldsymbol{\theta} + \alpha \boldsymbol{v}), \boldsymbol{y}^{(i)}) \right)$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$$

Note, gradient is evaluated after current velocity is applied.
Hence, Nesterov momentum is adding a correction factor to the standard method of momentum.

Algorithm 5: Stochastic Gradient Descent (SGD) with Nesterov Momentum

Require:
Learning rate ϵ_k, Momentum parameter α
Initial parameter $\boldsymbol{\theta}$, Initial velocity $\boldsymbol{v}$

while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Apply interim update: $\tilde{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} + \alpha \boldsymbol{v}$
Compute gradient (at interim point): $\boldsymbol{g} \leftarrow \frac{1}{m}\nabla_{\tilde{\boldsymbol{\theta}}} \sum_i L(f(\boldsymbol{x}^{(i)}; \tilde{\boldsymbol{\theta}}), \boldsymbol{y}^{(i)})$
Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \epsilon \boldsymbol{g}$
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
end while

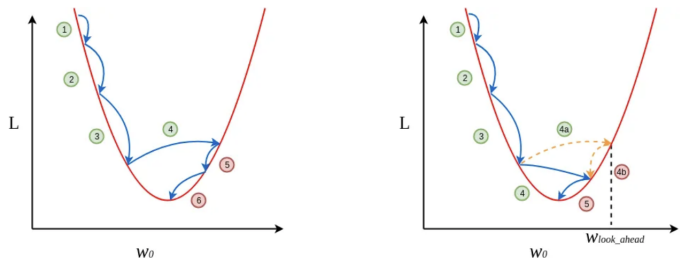


Figure 4: Momentum vs Nesterov Gradient Descent

Remark 4.3.36. *Parameter Initialisation Challenges*
Training deep neural networks is difficult as most algorithms are strongly affected by choice of initialisation.

Designing initialisation strategies is difficult as neural network optimisation is not yet well understood, as the nice properties from initialisation may not be preserved as training proceeds.
Some initial points may be beneficial for optimisation, but detrimental for generalisation.

Remark 4.3.37. *Parameter Initialisation Strategies*
Initial parameters need to 'break symmetry' between different units. May explicitly search for large set of basis functions mutually different from each other, but this may incur noticeable computational cost.
Random initialisation from high-entropy distribution over high-dimensional space is computationally cheaper.
Larger initial weights will yield stronger symmetry breaking effect and avoid redundant units. However, if weights are set too large, may result in chaos (extreme sensitivity to small perturbations of input). Exploding gradient may be mitigated by gradient clipping. The bias (i.e., parameters encoding conditional variance of a prediction) for each unit is set to heuristically chosen constraints.

Remark 4.3.38. *Weight Initialisation Heuristics*
Initialising parameters of neural network $\boldsymbol{\theta}$ to $\boldsymbol{\theta}_0$ is similar to imposing a Gaussian prior $p(\boldsymbol{\theta})$ with mean $\boldsymbol{\theta}_0$, hence it make sense to choose $\boldsymbol{\theta}_0$ near 0. Initialisation of $\boldsymbol{\theta}_0$ to large values means prior specifies which units should interact with each other, and how they should interact.
For a fully connected neural network with m inputs and n outputs, weights may be initialised by:

i. Sampling from $U\left(-\frac{1}{\sqrt{m}}, \frac{1}{\sqrt{m}}\right)$
ii. Sampling from normalised initialisation $W_{i,j} \sim U\left(-\sqrt{\frac{6}{m+n}}, \sqrt{\frac{6}{m+n}}\right)$ (Glorot and Bengio). Compromise between goal of initialising all layers to have same activation variance and that of same gradient variance. Derived from assumption that network is chain of matrix multiplications with no nonlinearities.
iii. Sampling from random orthogonal matrices, with scaling or gain factor g that accounts for nonlinearity at each layer, with specific values for different types of nonlinear activation functions. Assumes neural network is sequence of matrix multiplications without nonlinearities. Guarantees the total number of training iterations required to reach convergence is independent of depth.
Increasing scaling factor g pushes network toward regime where activations increase in norm in forward propagation, and gradients increase in norm in backward propagation.
For scaling rules that set all of initial weights to have same standard deviation, individual weight becomes extremely small when layers become large.
iv. Sparse initialisation (Martens), where each unit is initialised to have exactly k non-zero weight. Keep total number of input to unit independent from number of inputs m without making magnitude of individual weight elements shrink with m.
Imposes very strong prior on weights that are chosen to have large Gaussian values.

If computational resources allow, treat initial scale of weights for each layer as a hyper-parameter, and use hyper-parameter search algorithm. Choice of dense or sparse initialisation may also be a hyper-parameter.

Remark 4.3.39. *Bias Initialisation Heuristics*
Setting bias to zero is compatible with most weight initialisation schemes. The few other situations are:

i. If bias is for output unit, initialise bias to obtain right marginal statistics of the output. Assume initial weights are small enough that output of unit is determined only by bias. The bias will be inverse of activation function applied to marginal statistics of the output in training set.
If output is a highly skewed distribution over classes with marginal probability of class i given by element c_i of some vector $\boldsymbol{c}$, then bias vector $\boldsymbol{b}$ is such that $\text{softmax}(\boldsymbol{b}) = \boldsymbol{c}$. The models have output layers that should resemble input data $\boldsymbol{x}$, hence bias should match marginal distribution over $\boldsymbol{x}$.
ii. If bias is to be set to avoid causing too much saturation at initialisation (i.e., with ReLU). Not compatible with weight initialisation schemes that do not expect strong input form bias (i.e., with random walk init).
iii. If a unit controls whether other units are able to participate in a function. There is unit with output u and another unit $h \in [0, 1]$, multiplied together to produce output uh. Then h is a gate that determines if $uh \approx u$ or $uh \approx 0$. Hence, to set bias for h such that $h \approx 1$ at initialisation, else u does not learn.

Remark 4.3.40. *Variance or Precision Parameter Initialisation Heuristics*
For linear regression with conditional variance estimate using model

$$p(y \mid \boldsymbol{x}) = \mathcal{N}(y \mid \boldsymbol{w}^T\boldsymbol{x} + b, 1/\beta)$$

where β is a precision parameter. The variance of precision parameters may be initialised to 1.
Another approach is to assume initial weights are close enough to zero, and bias may be set while ignoring effect of the weights, then set bias to produce the correct marginal mean of the output, and set variance parameters to the marginal variance of the output in the training set.

Remark 4.3.41. *Model Parameter Initialisation with Machine Learning*
Initialise supervised model with parameters learned by unsupervised model trained on same inputs. Alternatively, perform supervised training on a related task, which may yield initialisation that offers faster convergence than random initialisation. Information on distribution is encoded into initial parameters of model.

Remark 4.3.42. *Adaptive Gradient (AdaGrad) Algorithm*
Algorithm individually adapts learning rates of all model parameters by scaling them inversely proportional to square root of sum of all of their historical squared values. Has greater progress in more gently sloped directions of parameter space. On deep neural networks, accumulation of squared gradients from beginning of training can result in premature and excessive decrease in effective learning rate.

Algorithm 6: Adaptive Gradient (AdaGrad) Algorithm

Require:
Global Learning rate ϵ
Initial parameter $\boldsymbol{\theta}$
Small constant δ, perhaps 10^{-7} for numerical stability

Initialise gradient accumulation variable $\boldsymbol{r} = \boldsymbol{0}$
while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \dots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Accumulate squared gradient: $\boldsymbol{r} \leftarrow \boldsymbol{r} + \boldsymbol{g} \odot \boldsymbol{g}$
Compute update: $\Delta\boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\delta + \sqrt{\boldsymbol{r}}} \odot \boldsymbol{g}$ (Division and square root applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

Remark 4.3.43. *Root Mean Squared Propagation (RMSProp) Algorithm*
Modifies AdaGrad to perform better in non-convex setting by changing gradient accumulation into exponentially weighted moving average, which discards history from extreme past so it can converge rapidly after finding a convex bowl. Use of moving average introduces a new hyper-parameter ρ that controls length scale of the moving average. RMSProp is effective and practical for deep neural networks.

Algorithm 7: Root Mean Squared Propagation (RMSProp) Algorithm

Require:
Global Learning rate ϵ
Decay rate ρ
Initial parameter $\boldsymbol{\theta}$
Small constant δ, perhaps 10^{-6} to stabilise division by small numbers

Initialise accumulation variable $\boldsymbol{r} = \boldsymbol{0}$
while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \dots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Accumulate squared gradient: $\boldsymbol{r} \leftarrow \rho\boldsymbol{r} + (1-\rho)\boldsymbol{g} \odot \boldsymbol{g}$
Compute update: $\Delta\boldsymbol{\theta} \leftarrow -\frac{\epsilon}{\sqrt{\delta + \boldsymbol{r}}} \odot \boldsymbol{g}$ ($\frac{\epsilon}{\sqrt{\delta + \boldsymbol{r}}}$ applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

Algorithm 8: Root Mean Squared Propagation (RMSProp) Algorithm

Require:
Global Learning rate ϵ
Decay rate ρ
Momentum coefficient α
Initial parameter $\boldsymbol{\theta}$
Initial velocity $\boldsymbol{v}$

Initialise accumulation variable $\boldsymbol{r} = \boldsymbol{0}$
while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \dots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute interim update $\tilde{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} + \alpha\boldsymbol{v}$
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\boldsymbol{\theta}}} \sum_i L(f(\boldsymbol{x}^{(i)}; \tilde{\boldsymbol{\theta}}), \boldsymbol{y}^{(i)})$

Accumulate gradient: $\boldsymbol{r} \leftarrow \rho \boldsymbol{r} + (1-\rho)\boldsymbol{g} \odot \boldsymbol{g}$
Compute velocity update: $\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \frac{\epsilon}{\sqrt{\boldsymbol{r}}} \odot \boldsymbol{g}$ ($\frac{1}{\sqrt{\boldsymbol{r}}}$ applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$
end while

Remark 4.3.44. *Adaptive Moment Estimation (Adam) Algorithm*
Variant on combination of RMSProp and momentum. Note, momentum is incorporated directly as an estimate of first order moment of gradient; bias corrections are made to estimates of both first-order moments and second-order moments to account for initialisation at origin.
Algorithm is fairly robust to choice of hyper-parameters.

Algorithm 9: Adaptive Moment Estimation (Adam) Algorithm

Require:
Step size ϵ (Suggested default: 0.001)
Exponential decay rates for moment estimates ρ_1, ρ_2 in $[0, 1)$ (Suggested defaults: 0.9 and 0.999)
Small constant δ used for numerical stabilisation (Suggested default: 10^{-8})
Initial parameters $\boldsymbol{\theta}$

Initialise 1st and 2nd momentum variables $\boldsymbol{s} = \boldsymbol{0}$, $\boldsymbol{r} = \boldsymbol{0}$
Initialise time step $t = 0$
while stopping criterion not met **do**
Sample a minibatch of m samples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with targets $\boldsymbol{y}^{(i)}$
Compute gradient: $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Update time step $t \leftarrow t + 1$
Update biased first and second moment estimate: $\boldsymbol{s} \leftarrow \rho_1 \boldsymbol{s} + (1-\rho_1)\boldsymbol{g}$ and $\boldsymbol{r} \leftarrow \rho_2 \boldsymbol{r} + (1-\rho_2)\boldsymbol{g} \odot \boldsymbol{g}$
Correct bias in first and second moment: $\hat{\boldsymbol{s}} \leftarrow \frac{\boldsymbol{s}}{1-\rho_1^t}$ and $\hat{\boldsymbol{r}} \leftarrow \frac{\boldsymbol{r}}{1-\rho_2^t}$
Compute update: $\Delta\boldsymbol{\theta} = -\epsilon \frac{\hat{\boldsymbol{s}}}{\sqrt{\hat{\boldsymbol{r}}} + \delta}$ (Operations applied element-wise)
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

Remark 4.3.45. *Newton's Method*
Based on second-order Taylor's series approximating $J(\boldsymbol{\theta})$ near the point $\boldsymbol{\theta}_0$, ignoring higher derivatives

$$J(\boldsymbol{\theta}) \approx J(\boldsymbol{\theta}_0) + (\boldsymbol{\theta} - \boldsymbol{\theta}_0)^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)^T \boldsymbol{H}(\boldsymbol{\theta} - \boldsymbol{\theta}_0)$$

where $\boldsymbol{H}$ is the Hessian of J with respect to $\boldsymbol{\theta}$ evaluated at $\boldsymbol{\theta}_0$.
Solving for critical point will get the Newton parameter update rule

$$\boldsymbol{\theta}^* = \boldsymbol{\theta}_0 - \boldsymbol{H}^{-1} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0)$$

Note that for deep learning, the surface of objective function is non-convex. Regularising the Hessian will solve many of the issues such as saddle points etc.

$$\boldsymbol{\theta}^* = \boldsymbol{\theta}_0 - [H(f(\boldsymbol{\theta}_0)) + \alpha \boldsymbol{I}]^{-1} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_0)$$

Due to requirement for inversion of the $k \times k$ matrix with time complexity $O(k^3)$ at every training iteration, only networks with very small number of parameters may be trained with this method.

Algorithm 10: Newton's Method with Objective $J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$

Require:
Initial parameter $\boldsymbol{\theta}_0$
Training set of m examples

while stopping criterion not met **do**
Compute gradient $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Compute Hessian: $\boldsymbol{H} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}}^2 \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Compute Hessian Inverse: $\boldsymbol{H}^{-1}$
Compute update: $\Delta\boldsymbol{\theta} = -\boldsymbol{H}^{-1}\boldsymbol{g}$
Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta\boldsymbol{\theta}$
end while

Remark 4.3.46. *Conjugate Gradient Method*
Method effectively avoids computation of inversion Hessian by descending conjugate directions, which is a search direction conjugate to the previous line search direction (will not undo progress made in that direction).
Let current and previous search directions be $\boldsymbol{d}_t, \boldsymbol{d}_{t-1}$. At training iteration t, the next search direction $\boldsymbol{d}_t$ is

$$\boldsymbol{d}_t = \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) + \beta_t \boldsymbol{d}_{t-1}$$

note that β_t may be computed via Fletcher Reeves:

$$\beta_t = \frac{\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)}{\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_{t-1})^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_{t-1})}$$

or may be computed with Polak-Ribiere:

$$\beta_t = \frac{(\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t) - \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_{t-1}))^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_t)}{\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_{t-1})^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}_{t-1})}$$

The non-linear conjugate gradient method is a variation which uses occasional resets where the method is restarted with line search along with the unaltered gradient.

Algorithm 11: Conjugate Gradient Method

Require:
Initial parameter $\boldsymbol{\theta}_0$
Training set of m examples
Initialise $\boldsymbol{\rho}_0 = 0$
Initialise $g_0 = 0$
Initialise $t = 1$

while stopping criterion not met **do**
Initialise the gradient $\boldsymbol{g}_t = \boldsymbol{0}$
Compute gradient $\boldsymbol{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
Compute Polak-Ribiere metric: $\beta_t = \frac{(\boldsymbol{g}_t - \boldsymbol{g}_{t-1})^T \boldsymbol{g}_t}{\boldsymbol{g}_{t-1}^T \boldsymbol{g}_{t-1}}$
(Nonlinear conjugate gradient: optionally reset $\beta_t = 0$, i.e., if t is a multiple of some constant k)
Compute search direction: $\boldsymbol{\rho}_t = -\boldsymbol{g}_t + \beta_t \boldsymbol{\rho}_{t-1}$
Perform line search to find: $\epsilon^* = \arg\min_\epsilon \frac{1}{m} \sum_{i=1}^m L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}_t + \epsilon \boldsymbol{\rho}_t), \boldsymbol{y}^{(i)})$
(On truly quadratic cost function, analytically solve for ϵ^*)
Apply update: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \epsilon^{\theta} \boldsymbol{\rho}_t$
Apply time step $t \leftarrow t + 1$
end while

Remark 4.3.47. *Nonlinear Conjugate Gradients*
Method of conjugate gradients is still applicable for non-linear objective function.
Conjugate directions are no longer assumed to remain at minimum of the objective for previous directions. Has occasional resets where method of conjugate gradients is restarted with line search along unaltered gradient.

Remark 4.3.48. *Broyden-Fletcher-Goldfarb-Shanno (BGFS) Algorithm*
Similar to conjugate gradient, but uses more direct approach to approximation of newton's update.
Method approximate the inverse of hessian $\boldsymbol{H}$ with matrix $\boldsymbol{M}_t$, iteratively refined by low rank updates to better approximate $\boldsymbol{H}^{-1}$. Direction of gradient descent is then determined by $\boldsymbol{\rho}_t = \boldsymbol{M}_t \boldsymbol{g}_t$. Line search is performed in this direction to determine step size ϵ^*. Final update of parameter is then $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \epsilon^* \boldsymbol{\rho}_t$.
Note, BGFS store matrix $\boldsymbol{M}$ which requires $O(n^2)$ memory, making it impractical for most modern models.

Remark 4.3.49. *Limited Memory BFGS (L-BFGS) Algorithm*
Memory costs of BFGS may be decreased by not storing complete inverse Hessian approximation $\boldsymbol{M}$. L-BFGS computes approximation $\boldsymbol{M}$ with same method as BFGS, but assumes at beginning that $\boldsymbol{M}^{(t-1)}$ is the identity matrix, rather than storing the approximation between steps.
If used with exact line searches, directions defined by L-BFGS are mutually conjugate, and this procedure remains well-behaved when minimum of line search is reached only approximately.
L-BFGS may be generalised to store some vectors used to update $\boldsymbol{M}$ at each time step, costing $O(n)$ per step.

Remark 4.3.50. *Meta-Algorithm: Batch Normalisation*
An update from optimisation algorithm may result in multiple-order effects, and in very deep networks, even higher-order interactions can be significant.

Batch normalisation provides an elegant way of re-parametrising any deep network, which reduces the problem of coordinating updates across multiple layers. Batch normalisation may be applied to any input/hidden layer. Let $\boldsymbol{H}$ be minibatch of activations of layer to normalise, with activations for each example appearing in a row. To normalise $\boldsymbol{H}$, replace it with $\boldsymbol{H}' = \frac{\boldsymbol{H}-\boldsymbol{\mu}}{\boldsymbol{\sigma}}$ where $\boldsymbol{\mu}$ is a vector of mean of each unit and σ is a vector of standard deviation of each unit. Then H_{ij} is normalised by subtracting μ_j and dividing σ_j. Rest of network then operates on $\boldsymbol{H}'$ just as in original network on $\boldsymbol{H}$.
At training, the mean and standard deviations are computed as

$$\boldsymbol{\mu} = \frac{1}{m}\sum_i \boldsymbol{H}_{i,;}, \quad \boldsymbol{\sigma} = \sqrt{\delta + \frac{1}{m}\sum_i (\boldsymbol{H}-\boldsymbol{\mu})_i^2}$$

where δ is a small positive value (i.e., 10^{-8}) to avoid undefined gradient of $\sqrt{z}$ at $z = 0$.
These are then back-propagated to normalise $\boldsymbol{H}$, hence gradient will never increase the standard deviation or mean of h_i. Batch normalisation re-parametrises the model to make some units always standardised.
At test time, μ and σ may be replaced by running averages from training time. This allows model to be evaluated on a single example, without needing to use definitions of μ and $\boldsymbol{\sigma}$ that depends on entire minibatch.

Remark 4.3.51. *Batch Normalisation but Maintain Network Express Power*
Replace batch of hidden activations $\boldsymbol{H}$ with $\boldsymbol{\gamma}\boldsymbol{H}' + \boldsymbol{\beta}$ rather than $\boldsymbol{H}'$, where $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ are learned parameters to allow new variable to have any mean and standard deviation. With original $\boldsymbol{H}'$, mean of $\boldsymbol{H}$ was determined by complicated interaction between parameters in layers below $\boldsymbol{H}$. With $\boldsymbol{\gamma}\boldsymbol{H}' + \boldsymbol{\beta}$, the mean is determined solely by $\boldsymbol{\beta}$, which is much easier to learn with gradient descent.
Most neural network layers are of form $\phi(\boldsymbol{X}\boldsymbol{W} + \boldsymbol{b})$, where ϕ is some fixed nonlinear activation function. Then $\boldsymbol{X}\boldsymbol{W} + \boldsymbol{b}$) should be replaced by normalised version of $\boldsymbol{X}\boldsymbol{W}$, and bias term omitted as it becomes redundant with β parameter applied from batch normalisation re-parametrisation. Statistics of input are non-Gaussian and less amenable to standardisation by linear operations.

Remark 4.3.52. *Meta-Algorithm: Coordinate Descent*
Solving optimisation problems by minimising the objective function $f(\boldsymbol{x})$ with respect to a single variable x_i, and rapidly cycling through all variables. *Block coordinate descent* refers to minimising with respect to a subset of variables simultaneously.
Coordinate descent works when different variables can be clearly separated into groups that play relatively isolated roles, or when optimisation with respect to one group of variables is significantly more efficient than optimisation with respect to all of the variables.

Example 4.3.53. *Sparse Encoding with Coordinate Descent*
Consider the cost function

$$J(\boldsymbol{H}, \boldsymbol{W}) = \sum_{i,j} |H_{ij}| + \sum_{i,j} (\boldsymbol{X} - \boldsymbol{W}^T\boldsymbol{H})_{i,j}^2$$

To find weight matrix $\boldsymbol{W}$ that can linearly decode a matrix of activation values $\boldsymbol{H}$ to reconstruct training set $\boldsymbol{X}$. Most applications also include weight decay or constraint on norms of columns of $\boldsymbol{W}$ to prevent the pathological solution with extremely small $\boldsymbol{H}$ and large $\boldsymbol{W}$.
Note J is not convex, but inputs may be divided into the dictionary parameters $\boldsymbol{W}$ and the code representations $\boldsymbol{H}$. Minimising object function with respect to either set is a convex problem.

Remark 4.3.54. *Polyak Averaging*
Average several points in trajectory through parameter space visited by optimisation algorithm.
If t iterations of gradient descent visit points $\boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(t)}$, then the output is $\hat{\boldsymbol{\theta}}^{(t)} = \frac{1}{t}\sum_i \boldsymbol{\theta}^{(i)}$.
The optimisation algorithm may leap back and forth across a valley several times without visiting the minimum point, and the average of all points on either side of the valley should be close to minimum value.
This is developed solely for stochastic gradient descent (SGD) to stabilise the estimates (**?**).
For non-convex problems, an exponentially decaying running average is used:

$$\hat{\boldsymbol{\theta}}^{(t)} = \alpha\hat{\boldsymbol{\theta}}^{(t-1)} + (1-\alpha)\boldsymbol{\theta}^{(t)}$$

Remark 4.3.55. *Supervised Pre-training*
Pre-training refers to training simple models on simple tasks before training the desired model on desired task. Greedy algorithms break problem into many components, then solve for optimal version of each component in isolation, which is must cheaper than algorithms that solve for best joint solution. Fine-tuning on joint optimisation algorithm searches for optimal solution to the full problem.
Greedy supervised training breaks supervised learning problems into other simpler supervised learning problems.

Transfer learning refers to pre-training a deep network on a set of tasks, then initialise a same-size network with first k layers of first net. The layers of second networks (upper layers initialised randomly) are then jointly trained to perform a different set of tasks with fewer training examples than for first set of tasks.
FitNets approach begins by training a network with low depth and high width, which becomes a teacher for a second network (the student), which is much deeper and thinner. Student network is trained to not only predict the output on original task, but also on value of middle layer of teacher network.

Remark 4.3.56. *Neural Network Design Choice: Linear Transformation*
Model units have generally moved towards using more linear functions, which allow model to have nice properties that make optimisation easier. The model's local gradient information corresponds reasonably well to moving towards a distant solution.

Remark 4.3.57. *Neural Network Design Choice: Linear Paths and Skip Connections*
Linear paths and skip connections between layers reduce length of shortest path from lower layer's parameters to the output, mitigating vanishing gradient problem.
Adding extra copies of output that are attached to intermediate hidden layers is an alternative to pre-training, where these 'auxiliary heads' are trained to perform the same task as primary output at the top of the network to ensure that the lower layers receive a large gradient. After training, these may be discarded.

Remark 4.3.58. *Continuation Methods*
Family of strategies that can make optimisation easier by choosing initial points to ensure that local optimisation spends most of its time in well-behaved regions. Method constructs series of objective functions $\{J^{(0)}, \dots, J^{(n)}\}$ which are increasingly difficult to minimise (well-behaved over less regions of $\boldsymbol{\theta}$ space) for the true cost function $J(\boldsymbol{\theta})$. The cost functions are designed such that a solution to one is a good initial point for the next.
Continuation methods aim to reach a global minimum despite many local minima. Easier cost functions are constructed through the blurring operation, which approximates

$$J^{(i)}(\boldsymbol{\theta}) = \mathbb{E}_{\theta' \sim \mathcal{N}(\boldsymbol{\theta}'; \boldsymbol{\theta}, \sigma^{(i)2})} J(\boldsymbol{\theta})'$$

via sampling. Some non-convex functions become approximately convex when blurred, but preserved enough information about local global minimum. However, approach might not work when

i. so many incremental cost functions are required that cost of entire procedure remains high;
ii. NP-hard optimisation problems remain NP-hard, even when continuation methods are applicable;
iii. function might not become convex, no matter how much it is blurred;
iv. function may become convex due to blurring, but minimum of this blurred function may track a local rather than global minimum of original cost function.

Remark 4.3.59. *Curriculum Learning*
Planning a learning process to begin by learning simple concepts, and progress to learning more complex topics that depend on these simpler concepts. Earlier objective functions $J^{(i)}$ are made easier by increasing the influence of simpler examples.
Stochastic curriculum is a variation of the approach where a random mix of easy and difficult examples is always presented to the learner, but average proportion of more difficult examples is gradually increased.

4.3.2 Convolutional Neural Networks

Definition 4.3.60. *Convolutional Neural Networks (CNNs)*
Neural network for processing data with a known, grid-like topology. Uses convolution instead of general matrix multiplication in at least one of their layers.

Definition 4.3.61. *Convolution*
The convolution of two functions f and g, denoted by $*$, is the integral of product of two functions after one is reflected about the y-axis and shifted.

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$

The first argument $f(\tau)$ is the input, and second argument $g(t-\tau)$ is the kernel. The output is feature map.

Definition 4.3.62. *Discrete Convolution*
For complex-valued functions f and g defined on set of $\mathbb{Z}$ integers, the discrete convolution of f and g is

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n-m]$$

If function g_N is periodic with period N, then the convolution is also periodic and identical to

$$(f * g_N)[n] = \sum_{m=0}^{N-1} \left(\sum_{k=-\infty}^{\infty} f[m+kN] \right) g_N[n-m]$$

Note that convolution is commutative, as the kernel is flipped relative to the input (as m increases, the index into input increases, but index into kernel decreases), but this is not an important property of neural network implementation. Neural network libraries implement cross-correlation instead.

Definition 4.3.63. *Cross-Correlation*
A measure of similarity of two series as a function of the displacement of one relative to the other. Also known as *sliding inner or dot product*. This is defined as

$$(f \star g)(\tau) = \int_{-\infty}^{\infty} \overline{f(\tau)}g(t+\tau)dt$$

This is related to convolution by

$$[f(t) \star gt](t) = [\overline{f(-t)} * g(t)](t)$$

Remark 4.3.64. *Motivation: Sparse Interactions*
The kernel is smaller than input, but is able to detect meaningful features. Hence fewer parameters are to be stored, reducing memory requirements of the model. Units in deeper layers may indirectly interact with a larger portion of the input, hence efficiently describe complicated interactions between many variables.

Remark 4.3.65. *Motivation: Parameter Sharing*
Each member of kernel is used at every position of the input except at boundary, which depends on design decisions regarding the boundary. Only one set of parameter is required to be learned. The runtime of forward propagation is at $O(k \times n)$ for $k \lll m$ connections for each input, given m input and n outputs.

Remark 4.3.66. *Motivation: Equivariant Representations*
An *equivariant* function is such that if the input changes, the output changes in the same way.
Applying a transformation, then convolution, is equivalent to applying convolution, then the transformation.

Remark 4.3.67. *Typical CNN Layers*
The layers of a typical CNN are:

i. Convolution: layer performs several convolutions in parallel to produce set of linear activations
ii. Detector: each linear activation is run through a non-linear activation function
iii. Pooling: modify the output of the layer further with summary statistics of nearby outputs

Example 4.3.68. *Example Types of Pooling*

i. Max Pooling: reports maximum output for each region of the feature map

ii. Average Pooling: reports average for each region of the feature map

iii. L^2 Pooling: calculates Euclidean norm (L^2 norm) for each region of the feature map

Remark 4.3.69. *Pooling on Invariance*
Pooling makes the representation approximately invariant to small translations of input.
Use of pooling is analogous to adding an infinitely strong prior that the function the layer learns must be invariant to small translations. As pooling summarises responses over whole neighbourhood, fewer pooling units than detector units may be used. To handle inputs of varying size, the size of offset between pooling regions may be changed so that the classification layer always receives the same number of summary statistics.

Remark 4.3.70. *Multi-Channel Operations*
Not commutative unless each operation has the same number of output and input channels.
Assume there exists 4-D kernel tensor $\mathcal{K}$ with element $K_{i,j,k,l}$ giving connection between strength between a unit in channel i of the output and a unit in channel j of the input, with offset of k rows and l columns between the output unit and input unit. Assume input consists of observed data $\mathcal{V}$ with element $V_{i,j,k}$ giving the value of input unit within channel i at row j and column k. Assume output consists of $\mathcal{Z}$ in same format as $\mathcal{V}$. If $\mathbb{Z}$ is produced by convolving $\mathcal{K}$ across $\mathcal{V}$ without flipping $\mathcal{K}$, then

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n}$$

where summation over l, m, n is over all values for which tensor indexing operations inside summation is valid.

Remark 4.3.71. *Downsampled Convolution*
To reduce computational cost by sampling only every s pixels (*stride*) in each direction of output, then

$$Z_{i,j,k} = c(\mathcal{K}, \mathcal{V}, s)_{i,j,k} = \sum_{l,m,n} [\mathcal{V}_{l,(j-1)\times s+m,(k-1)\times s+n} \mathcal{K}_{i,l,m,n}]$$

Remark 4.3.72. *Zero Padding*
Zero-padding is adding zeroes to end of input sequence $\mathcal{V}$ so that the total number of samples is equal to the next higher multiple of two.

i. Valid Convolution: no zero-padding used. If input image has width m and kernel has width k, the output will be width $m - k + 1$. Rate of shrinkage can be dramatic if kernel used are large.

ii. Same Convolution: just enough zero-padding is added to keep size of output equal to size of input. Border pixels may be underrepresented in the model.

iii. Full Convolution: zeroes are added for every pixel to be visited k times in each direction, resulting in an input image of width $m + k - 1$.

Remark 4.3.73. *Locally Connected Layers (Unshared Convolution)*
Similar operation to discrete convolution with small kernel, but without parameter sharing. Adjacency matrix of MLP is the same, but every connection has its own weight as specified by 6-D tensor $\mathcal{W}$, where the indices of $\mathcal{W}$ are the output channel i, the output row j, the output column k, the input channel l, the row offset within the the input m, and the column offset within the input n. The liner part of the layer is then

$$Z_{i,j,k} = \sum_{l,m,n} [\mathcal{V}_{l,j+m-1,k+n-1} w_{i,j,k,l,m,n}]$$

Locally connected layers are useful when it is known that each feature should be function of a small part of space but would not occur across all of space.

Remark 4.3.74. *Tiled Convolution*
Compromise between convolutional layer and locally connected layer. A set of kernels will be rotated through space. Immediate neighbouring locations will have different filters, but memory requirements for storing the parameters will increase only by a factor of the size of this set of kernels.
Let k be a 6-D tensor, where two of the dimensions correspond to different locations in output matrix. Output locations cycle through a set of t different choices of kernel stack in each direction.

$$Z_{i,j,k} = \sum_{l,m,n} \mathcal{V}_{l,j+m-1,k+n-1} \mathcal{K}_{i,l,m,n,j \bmod t+1,k \bmod t+1}$$